Python

Multiple Choice Questions — 466 Questions

Topics: Advanced Python | Built-ins | Closures | Collections | Comprehensions | Data Types | Decorators | Dicts | Exception Handling | File I/O | Functions | Generators | Inheritance | Iterators | Lists | Modules | OOP | Regular Expressions | Scope | Sets | Strings | Tuples

Total	Easy	Medium	Hard	With Code	Topics
466	138	187	141	188	22

MCQ: 186	Output: 96	Scenario: 92	Code Completion: 92

Instructions: Read each question carefully and select the ONE best answer. Questions marked **Output** ask what code prints/returns. **Code Completion** questions ask you to fill in the blank. **Scenario** questions describe a real problem to solve. Difficulty: **Easy** **Medium** **Hard**. The correct answer and explanation appear in green below each question.

Data Types

Easy

MCQ

Q1. Which of the following is an immutable data type in Python?

[OK] (A) tuple

(B) list

(C) dict

(D) set

Correct Answer: (A)

Explanation: Tuples are immutable sequences in Python. Once created, their elements cannot be changed, unlike lists, dicts, and sets.

Data Types

Q2. What is the result of type(3.14) in Python?

Easy

MCQ

[OK] (A)

(B)

(C)

(D)

Correct Answer: (A)
Explanation: Python uses float for decimal numbers. There is no built-in double type; Decimal is in the decimal module.

Data Types

Q3. Which function converts a string '42' to an integer?

Easy

MCQ

[OK] (A) int()

(B) str()

(C) float()

(D) chr()

Correct Answer: (A)
Explanation: int('42') parses the string and returns the integer 42. str() converts to string, float() to float, chr() to character.

Q4. What is the result of bool([]) in Python?

Data Types

Medium

MCQ

[OK] (A) FALSE

(B) TRUE

(C) None

(D) TypeError

Correct Answer: (A)
Explanation: Empty containers are falsy in Python. An empty list evaluates to False when converted to bool.

Q5. Which data type would you use to store a binary flag that is either on or off?

Data Types

Medium

MCQ

[OK] (A) bool

(B) int

(C) str

(D) bytes

Correct Answer: (A)
Explanation: bool is the natural choice for binary flags with values True/False. While int works, bool is semantically correct.

Data Types

Hard

MCQ

Q6. What is the output of isinstance(True, int) in Python?

[OK] (A) TRUE

(B) FALSE

(C) TypeError

(D) None

Correct Answer: (A)
Explanation: In Python, bool is a subclass of int. True is 1 and False is 0, so isinstance(True, int) returns True.

Data Types

Easy

MCQ

Q7. Which of the following creates a complex number in Python?

[OK] (A) 3 + 4j

(B) 3 + 4i

(C) complex(3, 4i)

(D) 3.4j

Correct Answer: (A)
Explanation: Python uses j (not i) for the imaginary part of complex numbers. 3 + 4j creates the complex number (3+4j).

Data Types

Medium

MCQ

Q8. What does sys.getsizeof(0) return relative to sys.getsizeof(2**30)?

[OK] (A) A smaller value

(B) The same value

(C) A larger value

(D) It raises an error

Correct Answer: (A)
*Explanation: Python integers have variable size. Larger integers require more memory, so getsizeof(0) < getsizeof(2**30).*

Strings

Easy

MCQ

Q9. Which method converts all characters of a string to uppercase?

[OK] (A) upper()

(B) capitalize()

(C) title()

(D) swapcase()

Correct Answer: (A)
Explanation: str.upper() converts all characters to uppercase. capitalize() only capitalizes the first character.

Q10. What does 'hello'.replace('l', 'r') return?

Strings

Easy

MCQ

[OK] (A) 'herro'

(B) 'herlo'

(C) 'hello'

(D) 'rerro'

Correct Answer: (A)
Explanation: replace() replaces ALL occurrences of the substring. Both 'l' characters are replaced with 'r'.

Q11. Which string method splits a string into a list based on a delimiter?

Strings

Medium

MCQ

[OK] (A) split()

(B) partition()

(C) slice()

(D) divide()

Correct Answer: (A)
Explanation: str.split() splits a string by the given delimiter (whitespace by default) and returns a list of substrings.

Q12. What is the output of 'python'[::-1]?

Strings

Medium

MCQ

[OK] (A) 'nohtyp'

(B) 'python'

(C) 'pytho'

(D) Error

Correct Answer: (A)
Explanation: The slice [::-1] reverses a string. Each character is taken in reverse order.

Q13. Which of the following is true about Python string interning?

Strings

Hard

MCQ

[OK] (A) Small strings and identifiers may be interned automatically

(B) All strings are interned

(C) Strings are never interned

(D) Only empty strings are interned

Correct Answer: (A)
Explanation: CPython automatically interns small strings and strings that look like identifiers for performance optimization.

Q14. What does the strip() method do?

Strings

Easy

MCQ

[OK] (A) Removes leading and trailing whitespace

(B) Removes all whitespace

(C) Removes the first character

(D) Removes the last character

Correct Answer: (A)
Explanation: strip() removes leading and trailing whitespace (or specified characters) from a string.

Q15. What is an f-string in Python?

Strings

Medium

MCQ

[OK] (A) A formatted string literal prefixed with f

(B) A frozen string

(C) A file string

(D) A function string

Correct Answer: (A)
Explanation: F-strings (f'...') allow embedding expressions inside string literals using curly braces, introduced in Python 3.6.

Strings

Q16. What does 'abc'.encode('utf-8') return?

Hard

MCQ

[OK] (A) b'abc'

(B) 'abc'

(C) [97, 98, 99]

(D) ('a', 'b', 'c')

Correct Answer: (A)
Explanation: encode() converts a string to bytes. For ASCII characters, the bytes literal looks like b'abc'.

Lists

Q17. Which method adds an element to the end of a list?

Easy

MCQ

[OK] (A) append()

(B) add()

(C) insert()

(D) push()

Correct Answer: (A)
Explanation: list.append(x) adds element x to the end of the list. insert() requires an index, and lists have no add() or push().

Q18. What does len([1, 2, 3]) return?

Lists

Easy

MCQ

[OK] (A) 3

(B) 2

(C) 4

(D) 1

Correct Answer: (A)
Explanation: len() returns the number of elements in a container. The list [1, 2, 3] has 3 elements.

Q19. What is the difference between list.sort() and sorted(list)?

Lists

Medium

MCQ

[OK] (A) sort() modifies in-place and returns None; sorted() returns a new list

(B) They are identical

(C) sorted() modifies in-place

(D) sort() returns a new list

Correct Answer: (A)
Explanation: list.sort() sorts in-place and returns None. sorted() creates and returns a new sorted list, leaving the original unchanged.

Q20. What does [1, 2, 3] + [4, 5] produce?

Lists

Medium

MCQ

[OK] (A) [1, 2, 3, 4, 5]

(B) [5, 7, 3]

(C) Error

(D) [[1,2,3],[4,5]]

Correct Answer: (A)
Explanation: The + operator concatenates two lists, creating a new list with all elements from both lists.

Q21. What is the time complexity of list.pop(0) in Python?

Lists

Hard

MCQ

[OK] (A) O(n)

(B) O(1)

(C) O(log n)

(D) O(n^2)

Correct Answer: (A)
Explanation: Removing the first element requires shifting all remaining elements, making it O(n). Use collections.deque for O(1) popleft.

Q22. How do you create an empty list in Python?

Lists

Easy

MCQ

[OK] (A) [] or list()

(B) () or tuple()

(C) {} or dict()

(D) new List()

Correct Answer: (A)
Explanation: An empty list is created with [] literal syntax or by calling list() with no arguments.

Q23. What does list.extend([4, 5]) do compared to list.append([4, 5])?

Lists

Medium

MCQ

[OK] (A) extend adds each element individually; append adds the list as one element

(B) They are identical

(C) extend adds as one element; append adds individually

(D) Both raise errors

Correct Answer: (A)
Explanation: extend() iterates over the argument and adds each element. append() adds the entire argument as a single element.

Lists

Hard

MCQ

Q24. What is the result of a = [1]; b = a; b.append(2); print(a)?

[OK] (A) [1, 2]

(B) [1]

(C) [2]

(D) Error

Correct Answer: (A)
Explanation: b = a does not copy the list; both variables reference the same object. Modifying b also modifies a.

Dicts

Easy

MCQ

Q25. How do you access the value associated with key 'name' in a dictionary d?

[OK] (A) d['name']

(B) d.name

(C) d.get_value('name')

(D) d->name

Correct Answer: (A)
Explanation: Dictionary values are accessed using bracket notation d['key']. d.get('name') also works with a default.

Q26. Which method returns all keys of a dictionary?

Dicts

Easy

MCQ

[OK] (A) keys()

(B) values()

(C) items()

(D) get()

Correct Answer: (A)
Explanation: dict.keys() returns a view object containing all keys. values() returns values, items() returns key-value pairs.

Q27. What happens when you access a key that does not exist using d[key]?

Dicts

Medium

MCQ

[OK] (A) KeyError is raised

(B) None is returned

(C) 0 is returned

(D) An empty string is returned

Correct Answer: (A)
Explanation: Accessing a missing key with bracket notation raises KeyError. Use d.get(key) to return None instead.

Q28. What does dict.setdefault('key', []) do?

Dicts

Medium

MCQ

[OK] (A) Returns value if key exists, otherwise sets key to [] and returns it

(B) Always sets the key to []

(C) Deletes the key

(D) Raises an error if key exists

Correct Answer: (A)

Explanation: setdefault returns the existing value if the key is present; otherwise it inserts the key with the default value and returns it.

Q29. In Python 3.7+, what is guaranteed about dictionary iteration order?

Dicts

Hard

MCQ

[OK] (A) Insertion order is preserved

(B) Keys are sorted alphabetically

(C) Order is random

(D) Order depends on hash values

Correct Answer: (A)

Explanation: Since Python 3.7, dictionaries maintain insertion order as part of the language specification (CPython did this since 3.6).

Dicts

Easy

MCQ

Q30. Which method safely retrieves a value from a dict with a default?

[OK] (A) get()

(B) fetch()

(C) retrieve()

(D) find()

Correct Answer: (A)
Explanation: dict.get(key, default) returns the value for key if it exists, otherwise returns the default (None if not specified).

Dicts

Hard

MCQ

Q31. What is the average time complexity of dictionary lookup in Python?

[OK] (A) O(1)

(B) O(n)

(C) O(log n)

(D) O(n^2)

Correct Answer: (A)
Explanation: Python dicts use hash tables, providing O(1) average-case lookup. Worst case is O(n) due to hash collisions.

Q32. Which of the following can NOT be used as a dictionary key?

Dicts

Medium

MCQ

[OK] (A) [1, 2, 3]

(B) (1, 2, 3)

(C) 42

(D) 'hello'

Correct Answer: (A)
Explanation: Lists are mutable and not hashable, so they cannot be dictionary keys. Tuples, ints, and strings are hashable.

Q33. Which of the following correctly creates a set in Python?

Sets

Easy

MCQ

[OK] (A) {1, 2, 3}

(B) [1, 2, 3]

(C) (1, 2, 3)

(D) set[1, 2, 3]

Correct Answer: (A)
Explanation: Curly braces with values create a set. Note: {} creates an empty dict, not an empty set; use set() for that.

Q34. Which method adds an element to a set?

Sets

Easy

MCQ

[OK] (A) add()

(B) append()

(C) insert()

(D) push()

Correct Answer: (A)
Explanation: set.add(x) adds element x to the set. Sets do not have append() or insert() methods.

Q35. What does set_a & set_b return?

Sets

Medium

MCQ

[OK] (A) The intersection of set_a and set_b

(B) The union

(C) The difference

(D) A boolean

Correct Answer: (A)
Explanation: The & operator computes the intersection of two sets, returning elements common to both.

Q36. Can a set contain duplicate elements in Python?

Sets

Medium

MCQ

[OK] (A) No, duplicates are automatically removed

(B) Yes, duplicates are allowed

(C) Only if they are strings

(D) Only with a special flag

Correct Answer: (A)
Explanation: Sets are collections of unique elements. Adding a duplicate element has no effect.

Q37. What is a frozenset in Python?

Sets

Hard

MCQ

[OK] (A) An immutable version of a set

(B) A set stored on disk

(C) A sorted set

(D) A set with fixed capacity

Correct Answer: (A)
Explanation: frozenset is an immutable set that can be used as a dictionary key or as an element of another set.

Q38. What does set_a - set_b compute?

Sets

Medium

MCQ

[OK] (A) Elements in set_a but not in set_b

(B) Elements in both sets

(C) Elements in either set

(D) The symmetric difference

Correct Answer: (A)
Explanation: The - operator computes the set difference: elements present in the left set but not in the right set.

Q39. What is the time complexity of the 'in' operator for a set vs a list?

Sets

Hard

MCQ

[OK] (A) O(1) for set, O(n) for list

(B) O(n) for both

(C) O(1) for both

(D) O(log n) for set, O(n) for list

Correct Answer: (A)
Explanation: Sets use hash tables for O(1) average membership testing. Lists require linear search, giving O(n).

Q40. How do you create a single-element tuple?

Tuples

Easy

MCQ

[OK] (A) (1,)

(B) -1

(C) [1]

(D) tuple(1)

Correct Answer: (A)
Explanation: A trailing comma is required for single-element tuples. (1) is just the integer 1 in parentheses.

Q41. What happens if you try to modify an element of a tuple?

Tuples

Easy

MCQ

[OK] (A) TypeError is raised

(B) The element is modified

(C) A new tuple is created

(D) Nothing happens

Correct Answer: (A)
Explanation: Tuples are immutable. Attempting to assign to an index raises TypeError.

Q42. What is tuple unpacking?

Tuples

Medium

MCQ

[OK] (A) Assigning tuple elements to multiple variables simultaneously

(B) Converting a tuple to a list

(C) Adding elements to a tuple

(D) Removing elements from a tuple

Correct Answer: (A)
Explanation: Tuple unpacking (e.g., a, b, c = (1, 2, 3)) assigns each element to a corresponding variable.

Q43. Which method returns the number of occurrences of a value in a tuple?

Tuples

Medium

MCQ

[OK] (A) count()

(B) frequency()

(C) occurrences()

(D) tally()

Correct Answer: (A)
Explanation: tuple.count(x) returns the number of times x appears in the tuple. It is one of only two tuple methods (with index).

Q44. Why are tuples generally faster than lists in Python?

Tuples

Hard

MCQ

[OK] (A) Tuples are stored more compactly and have immutability optimizations

(B) Tuples use linked lists internally

(C) Tuples are compiled to C code

(D) There is no performance difference

Correct Answer: (A)
Explanation: Tuples have a smaller memory footprint and CPython caches small tuples. Immutability enables compiler optimizations.

Q45. Which keyword is used to define a function in Python?

Functions

Easy

MCQ

[OK] (A) def

(B) function

(C) func

(D) define

Correct Answer: (A)
Explanation: The 'def' keyword is used to define functions in Python, followed by the function name and parameters.

Q46. What does a function return if there is no return statement?

Functions

Easy

MCQ

[OK] (A) None

(B) 0

(C) Empty string

(D) FALSE

Correct Answer: (A)
Explanation: If a function has no return statement or a bare return, it implicitly returns None.

Q47. What does *args represent in a function definition?

Functions

Medium

MCQ

[OK] (A) Variable-length positional arguments as a tuple

(B) Variable-length keyword arguments

(C) A required argument

(D) A list of strings

Correct Answer: (A)
*Explanation: *args collects extra positional arguments into a tuple, allowing the function to accept any number of positional arguments.*

Q48. What does **kwargs represent in a function definition?

Functions

Medium

MCQ

[OK] (A) Variable-length keyword arguments as a dictionary

(B) Variable-length positional arguments

(C) A required keyword argument

(D) A frozen dictionary

Correct Answer: (A)
*Explanation: **kwargs collects extra keyword arguments into a dictionary, allowing the function to accept any named arguments.*

Q49. What is the danger of using a mutable default argument in Python?

Functions

Hard

MCQ

[OK] (A) The default is shared across all calls, leading to unexpected accumulation

(B) It causes a syntax error

(C) It creates a new object each call

(D) It only works with lists

Correct Answer: (A)
Explanation: Mutable default arguments are evaluated once at function definition time. Modifications persist across calls.

Q50. What is a lambda function in Python?

Functions

Medium

MCQ

[OK] (A) An anonymous single-expression function

(B) A function that runs in parallel

(C) A recursive function

(D) A generator function

Correct Answer: (A)
Explanation: Lambda functions are small anonymous functions defined with the lambda keyword, limited to a single expression.

Q51. What is the purpose of the functools.lru_cache decorator?

Functions

Hard

MCQ

[OK] (A) It caches function results to avoid recomputation

(B) It limits function calls per second

(C) It logs function usage

(D) It runs functions lazily

Correct Answer: (A)
Explanation: lru_cache memoizes function results based on arguments, using a Least Recently Used eviction policy.

Functions

Q52. How do you call a function named greet with argument 'Alice'?

Easy

MCQ

[OK] (A) greet('Alice')

(B) call greet('Alice')

(C) greet 'Alice'

(D) invoke greet('Alice')

Correct Answer: (A)
Explanation: Functions are called by their name followed by parentheses containing any arguments.

OOP

Q53. Which keyword is used to create a class in Python?

Easy

MCQ

[OK] (A) class

(B) Class

(C) struct

(D) object

Correct Answer: (A)
Explanation: The 'class' keyword (lowercase) is used to define a class in Python.

Q54. What is the first parameter of an instance method in Python?

OOP

Easy

MCQ

[OK] (A) self

(B) this

(C) cls

(D) instance

Correct Answer: (A)
Explanation: By convention, the first parameter of instance methods is 'self', referring to the instance itself.

Q55. What is the purpose of __init__ in a Python class?

OOP

Medium

MCQ

[OK] (A) It initializes a new instance of the class

(B) It destroys an instance

(C) It creates a class variable

(D) It defines the class name

Correct Answer: (A)
Explanation: __init__ is the constructor method called automatically when creating a new instance, used to initialize attributes.

Q56. What is encapsulation in OOP?

OOP

Medium

MCQ

[OK] (A) Bundling data and methods together and restricting direct access

(B) Inheriting from multiple classes

(C) Creating abstract classes

(D) Using global variables

Correct Answer: (A)
Explanation: Encapsulation bundles data (attributes) and methods within a class, using access control to protect internal state.

Q57. What is the difference between __str__ and __repr__ in Python?

OOP

Hard

MCQ

[OK] (A) __str__ is for end-user display; __repr__ is for developers and debugging

(B) They are identical

(C) __repr__ is for display; __str__ is for debugging

(D) __str__ cannot be overridden

Correct Answer: (A)
Explanation: __str__ returns a human-readable string, while __repr__ returns an unambiguous developer-oriented representation.

Q58. What are Python descriptors?

OOP

Hard

MCQ

[OK] (A) Objects that define __get__, __set__, or __delete__ to customize attribute access

(B) Functions that describe a class

(C) Docstrings attached to methods

(D) Type annotations

Correct Answer: (A)
Explanation: Descriptors implement the descriptor protocol (__get__, __set__, __delete__) to customize attribute access on classes.

Q59. What does the @classmethod decorator do?

OOP

Medium

MCQ

[OK] (A) Binds the method to the class, receiving cls as first argument

(B) Makes the method private

(C) Makes the method static

(D) Prevents method overriding

Correct Answer: (A)
Explanation: @classmethod binds the method to the class rather than the instance. It receives the class (cls) as its first argument.

Q60. What does the @staticmethod decorator do?

OOP

Medium

MCQ

[OK] (A) Defines a method that does not receive self or cls

(B) Makes the method unchangeable

(C) Binds the method to the class

(D) Creates a class variable

Correct Answer: (A)
Explanation: @staticmethod defines a method that belongs to the class namespace but does not receive self or cls automatically.

Q61. How does a class Dog inherit from a class Animal in Python?

Inheritance

Easy

MCQ

[OK] (A) class Dog(Animal):

(B) class Dog extends Animal:

(C) class Dog inherits Animal:

(D) class Dog <- Animal:

Correct Answer: (A)
Explanation: In Python, inheritance is specified by putting the parent class in parentheses after the class name.

Q62. What is the MRO in Python?

Inheritance

Medium

MCQ

[OK] (A) Method Resolution Order - the order in which base classes are searched

(B) Multiple Return Objects

(C) Module Reference Order

(D) Method Repeat Override

Correct Answer: (A)
Explanation: MRO determines the order in which methods are looked up in the class hierarchy, using the C3 linearization algorithm.

Q63. What does super() do in Python?

Inheritance

Medium

MCQ

[OK] (A) Returns a proxy object that delegates method calls to a parent class

(B) Creates a superclass

(C) Overrides all parent methods

(D) Prevents inheritance

Correct Answer: (A)
Explanation: super() returns a proxy object that delegates method calls to the parent class, following the MRO.

Inheritance

Hard

MCQ

Q64. What is the diamond problem in multiple inheritance?

[OK] (A) Ambiguity when a class inherits from two classes that share a common base

(B) A class inheriting from itself

(C) Circular imports

(D) A class with four parents

Correct Answer: (A)
Explanation: The diamond problem occurs when two parent classes share a common ancestor, creating ambiguity. Python solves this with C3 linearization.

Inheritance

Easy

MCQ

Q65. Can a Python class inherit from multiple parent classes?

[OK] (A) Yes, Python supports multiple inheritance

(B) No, only single inheritance is allowed

(C) Only with a special keyword

(D) Only abstract classes can

Correct Answer: (A)
Explanation: Python supports multiple inheritance. A class can list multiple parent classes separated by commas.

Q66. What is an Abstract Base Class (ABC) in Python?

Inheritance

Hard

MCQ

[OK] (A) A class that cannot be instantiated and may have abstract methods

(B) A class with no methods

(C) A compiled C class

(D) A class that only has static methods

Correct Answer: (A)
Explanation: ABCs from the abc module define interfaces. They cannot be instantiated directly and force subclasses to implement abstract methods.

Q67. What is a decorator in Python?

Decorators

Medium

MCQ

[OK] (A) A function that modifies the behavior of another function

(B) A comment style

(C) A type annotation

(D) A class attribute

Correct Answer: (A)
Explanation: Decorators are functions that take a function as input and return a modified version, applied with @decorator syntax.

Q68. What does @property do in Python?

Decorators

Medium

MCQ

[OK] (A) Defines a method that can be accessed like an attribute

(B) Makes an attribute private

(C) Creates a class property file

(D) Locks an attribute value

Correct Answer: (A)
Explanation: @property turns a method into a getter, allowing attribute-like access while running code behind the scenes.

Q69. What does functools.wraps do when writing a decorator?

Decorators

Hard

MCQ

[OK] (A) Preserves the original function's metadata (name, docstring, etc.)

(B) Wraps the function in a try-except

(C) Makes the decorator reusable

(D) Converts the function to a generator

Correct Answer: (A)
Explanation: functools.wraps copies the wrapped function's __name__, __doc__, and other attributes to the wrapper function.

Q70. How can a decorator accept arguments?

Decorators

Hard

MCQ

[OK] (A) By using a decorator factory: a function that returns the actual decorator

(B) Decorators cannot accept arguments

(C) By using global variables

(D) By modifying sys.argv

Correct Answer: (A)
Explanation: A decorator factory is a function that takes arguments and returns a decorator. Example: @repeat(3) uses repeat as a factory.

Q71. Which of the following is equivalent to @decorator on a function f?

Decorators

Medium

MCQ

[OK] (A) f = decorator(f)

(B) f = f(decorator)

(C) decorator = f

(D) f.decorator()

Correct Answer: (A)
Explanation: @decorator above a function definition is syntactic sugar for f = decorator(f) after the function is defined.

Q72. What keyword makes a function a generator function?

Generators

Medium

MCQ

[OK] (A) yield

(B) return

(C) generate

(D) next

Correct Answer: (A)
Explanation: The presence of yield in a function body makes it a generator function. Calling it returns a generator iterator.

Q73. What is the main advantage of generators over lists?

Generators

Medium

MCQ

[OK] (A) They produce values lazily, saving memory

(B) They are faster to iterate

(C) They support indexing

(D) They are thread-safe

Correct Answer: (A)
Explanation: Generators produce values one at a time on demand, avoiding storing the entire sequence in memory.

Generators

Q74. What does the send() method do on a generator?

Hard

MCQ

[OK] (A) Sends a value into the generator, resuming it and providing the value to yield

(B) Sends the generator to another thread

(C) Closes the generator

(D) Resets the generator

Correct Answer: (A)
Explanation: gen.send(value) resumes the generator and the yield expression evaluates to value, enabling two-way communication.

Generators

Q75. What does 'yield from' do in Python?

Hard

MCQ

[OK] (A) Delegates to a sub-generator, forwarding sent values and exceptions

(B) Yields a value from a list

(C) Creates a generator from a file

(D) Imports a generator from a module

Correct Answer: (A)
Explanation: 'yield from iterable' delegates iteration to the sub-iterable, transparently forwarding send() and throw() calls.

Generators

Easy

MCQ

Q76. What does next() do when called on a generator?

[OK] (A) Retrieves the next yielded value

(B) Skips to the end

(C) Resets the generator

(D) Returns all remaining values

Correct Answer: (A)
Explanation: next(gen) resumes the generator until the next yield, returning the yielded value. Raises StopIteration when exhausted.

Iterators

Medium

MCQ

Q77. Which two methods must an iterator implement?

[OK] (A) __iter__ and __next__

(B) __init__ and __del__

(C) __get__ and __set__

(D) __enter__ and __exit__

Correct Answer: (A)
Explanation: The iterator protocol requires __iter__ (returning self) and __next__ (returning the next value or raising StopIteration).

Iterators

Easy

MCQ

Q78. What built-in function creates an iterator from an iterable?

[OK] (A) iter()

(B) next()

(C) list()

(D) range()

Correct Answer: (A)
Explanation: iter(iterable) returns an iterator object. You can then use next() to get successive elements.

Iterators

Medium

MCQ

Q79. What exception signals the end of an iterator?

[OK] (A) StopIteration

(B) EndOfIterator

(C) IteratorError

(D) IndexError

Correct Answer: (A)
Explanation: When an iterator is exhausted, __next__() raises StopIteration to signal there are no more elements.

Iterators

Q80. What is the difference between an iterable and an iterator?

Hard

MCQ

[OK] (A) An iterable has __iter__ returning an iterator; an iterator also has __next__

(B) They are the same thing

(C) Iterables are faster

(D) Iterators support indexing

Correct Answer: (A)
Explanation: An iterable implements __iter__() which returns an iterator. An iterator implements both __iter__() and __next__().

Comprehensions

Q81. What is a list comprehension in Python?

Easy

MCQ

[OK] (A) A concise way to create lists using [expr for item in iterable]

(B) A method to understand lists

(C) A way to compress lists

(D) A list sorting technique

Correct Answer: (A)
Explanation: List comprehensions provide a compact syntax for creating lists from existing iterables with optional filtering.

Comprehensions

Easy

MCQ

Q82. Which of the following is a valid dict comprehension?

[OK] (A) {k: v for k, v in pairs}

(B) [k: v for k, v in pairs]

(C) (k: v for k, v in pairs)

(D)

Correct Answer: (A)
Explanation: Dict comprehensions use curly braces with key: value syntax inside the comprehension expression.

Comprehensions

Medium

MCQ

Q83. What does (x**2 for x in range(5)) create?

[OK] (A) A generator expression

(B) A tuple

(C) A list

(D) A set

Correct Answer: (A)
Explanation: Parentheses around a comprehension-like expression create a generator expression, not a tuple.

Comprehensions

Medium

MCQ

Q84. How do you add a condition to a list comprehension?

[OK] (A) [x for x in lst if condition]

(B) [x for x in lst where condition]

(C) [x if condition for x in lst]

(D) [x for x in lst when condition]

Correct Answer: (A)
Explanation: The if clause comes after the for clause to filter elements. [x for x in lst if x > 0] keeps only positive values.

Comprehensions

Hard

MCQ

Q85. What is a nested list comprehension?

[OK] (A) A comprehension with multiple for clauses to handle nested iterations

(B) A list inside a list

(C) A comprehension that calls itself

(D) A comprehension using recursion

Correct Answer: (A)
*Explanation: Nested comprehensions have multiple for clauses, e.g., [x*y for x in range(3) for y in range(3)] creates a flat list.*

Exception Handling

Q86. Which keyword is used to catch exceptions in Python?

Easy

MCQ

[OK] (A) except

(B) catch

(C) handle

(D) rescue

Correct Answer: (A)
Explanation: Python uses try/except blocks. The 'except' keyword catches specified exceptions.

Exception Handling

Q87. What does the finally block do?

Easy

MCQ

[OK] (A) Executes code regardless of whether an exception occurred

(B) Catches all exceptions

(C) Raises an exception

(D) Terminates the program

Correct Answer: (A)
Explanation: The finally block always executes after try/except, making it ideal for cleanup (closing files, connections).

Exception Handling

Medium

MCQ

Q88. What does the 'raise' keyword do?

[OK] (A) Manually raises an exception

(B) Catches an exception

(C) Ignores an exception

(D) Logs an exception

Correct Answer: (A)
Explanation: 'raise' is used to explicitly throw an exception. 'raise ValueError(msg)' creates and raises a ValueError.

Exception Handling

Medium

MCQ

Q89. What is the purpose of the 'else' clause in a try/except block?

[OK] (A) Executes only if no exception was raised in the try block

(B) Catches remaining exceptions

(C) Always executes

(D) Executes if an exception was raised

Correct Answer: (A)
Explanation: The else clause runs only when the try block completes without raising any exception.

Exception Handling

Q90. How do you create a custom exception class in Python?

Hard

MCQ

[OK] (A) By inheriting from Exception or a subclass of Exception

(B) By using the error keyword

(C) By modifying sys.exceptions

(D) By writing a config file

Correct Answer: (A)
Explanation: Custom exceptions are created by subclassing Exception: class MyError(Exception): pass.

Exception Handling

Q91. What is exception chaining in Python 3?

Hard

MCQ

[OK] (A) Using 'raise X from Y' to link a new exception to its cause

(B) Catching multiple exceptions in one except

(C) Raising exceptions in a loop

(D) Using nested try blocks

Correct Answer: (A)
Explanation: 'raise X from Y' sets Y as the __cause__ of X, preserving the chain of exceptions for debugging.

File I/O

Easy

MCQ

Q92. Which built-in function opens a file in Python?

[OK] (A) open()

(B) file()

(C) read()

(D) load()

Correct Answer: (A)
Explanation: open(filename, mode) is the built-in function to open files. It returns a file object.

File I/O

Easy

MCQ

Q93. What is the recommended way to handle files in Python?

[OK] (A) Using a with statement (context manager)

(B) Using try/finally

(C) Calling close() manually

(D) Using global file handles

Correct Answer: (A)
Explanation: The 'with' statement ensures proper resource cleanup by automatically closing the file when the block exits.

File I/O

Medium

MCQ

Q94. What mode opens a file for writing, creating it if it does not exist?

[OK] (A) 'w'

(B) 'r'

(C) 'x'

(D) 'a'

Correct Answer: (A)
Explanation: 'w' mode opens for writing, truncating the file if it exists or creating it if it does not.

File I/O

Medium

MCQ

Q95. What is the difference between 'r' and 'rb' modes?

[OK] (A) 'r' opens in text mode; 'rb' opens in binary mode

(B) 'r' is read-only; 'rb' is read-write

(C) 'rb' is faster

(D) There is no difference

Correct Answer: (A)
Explanation: 'r' opens in text mode with encoding/decoding. 'rb' opens in binary mode, returning bytes objects.

File I/O

Hard

MCQ

Q96. What does the pathlib module provide over os.path?

[OK] (A) Object-oriented filesystem paths with methods and operator overloading

(B) Faster file operations

(C) Network file access

(D) Database file storage

Correct Answer: (A)
Explanation: pathlib provides Path objects with methods like .read_text(), operator / for joining, and cleaner cross-platform path handling.

Modules

Easy

MCQ

Q97. What keyword is used to import a module in Python?

[OK] (A) import

(B) include

(C) require

(D) using

Correct Answer: (A)
Explanation: The 'import' keyword loads a module. You can also use 'from module import name' for specific objects.

Q98. What is the purpose of __init__.py in a directory?

Modules

Medium

MCQ

[OK] (A) It marks the directory as a Python package

(B) It initializes variables

(C) It runs first when Python starts

(D) It is required for all directories

Correct Answer: (A)

Explanation: __init__.py makes a directory a Python package. In Python 3, namespace packages work without it, but it is still common.

Q99. What does 'if __name__ == "__main__":' guard against?

Modules

Medium

MCQ

[OK] (A) Code running when the module is imported (it only runs when executed directly)

(B) Syntax errors

(C) Import errors

(D) Name collisions

Correct Answer: (A)

Explanation: __name__ is '__main__' only when the script is run directly. When imported, it equals the module name.

Q100. How does Python's module caching work?

Modules

Hard

MCQ

[OK] (A) Modules are cached in sys.modules after first import

(B) Modules are reloaded every import

(C) Modules are cached on disk

(D) There is no caching

Correct Answer: (A)
Explanation: After a module is first imported, Python caches it in sys.modules. Subsequent imports use the cached version.

Q101. What does 'from math import sqrt' do?

Modules

Easy

MCQ

[OK] (A) Imports only the sqrt function from the math module

(B) Imports the entire math module

(C) Creates a new sqrt function

(D) Renames the math module

Correct Answer: (A)
Explanation: 'from module import name' imports a specific object from a module directly into the current namespace.

Regular Expressions

Q102. Which module provides regular expression support in Python?

Easy

MCQ

[OK] (A) re

(B) regex

(C) regexp

(D) pattern

Correct Answer: (A)
Explanation: The 're' module is Python's built-in regular expression library. The third-party 'regex' module offers extended features.

Regular Expressions

Q103. What does re.findall(pattern, string) return?

Medium

MCQ

[OK] (A) A list of all non-overlapping matches

(B) The first match only

(C) A boolean

(D) A compiled pattern

Correct Answer: (A)
Explanation: re.findall() returns a list of all non-overlapping matches of the pattern in the string.

Regular Expressions

Q104. What does the \d pattern match in a regular expression?

Medium

MCQ

[OK] (A) Any digit character

(B) Any non-digit character

(C) Any whitespace

(D) The letter d

Correct Answer: (A)
Explanation: \d matches any Unicode digit (equivalent to [0-9] for ASCII). \D matches non-digits.

Regular Expressions

Q105. What is a lookahead assertion in regex?

Hard

MCQ

[OK] (A) A pattern that matches only if followed by another pattern, without consuming it

(B) A way to look at previous matches

(C) A method to speed up matching

(D) A named group

Correct Answer: (A)
Explanation: Lookahead (?=...) asserts that what follows matches the pattern but does not include it in the match result.

Regular Expressions

Q106. What is the difference between re.match() and re.search()?

Hard

MCQ

[OK] (A) match() checks only at the start of the string; search() checks anywhere

(B) They are identical

(C) search() checks only at the start

(D) match() is case-insensitive

Correct Answer: (A)
Explanation: re.match() anchors to the beginning of the string. re.search() scans through the entire string for a match.

Built-ins

Q107. What does the map() function do?

Easy

MCQ

[OK] (A) Applies a function to each item of an iterable

(B) Creates a dictionary

(C) Sorts a list

(D) Maps keys to values

Correct Answer: (A)
Explanation: map(func, iterable) returns an iterator that applies func to every item of the iterable.

Q108. What does the filter() function do?

Built-ins

Easy

MCQ

[OK] (A) Returns elements from an iterable where the function returns True

(B) Removes None values only

(C) Sorts elements

(D) Converts data types

Correct Answer: (A)
Explanation: filter(func, iterable) returns an iterator yielding elements for which func returns True.

Q109. What does zip() do in Python?

Built-ins

Medium

MCQ

[OK] (A) Creates an iterator of tuples pairing elements from multiple iterables

(B) Compresses files

(C) Merges dictionaries

(D) Concatenates strings

Correct Answer: (A)
Explanation: zip() pairs up elements from multiple iterables. It stops at the shortest iterable.

Q110. What does enumerate() return?

Built-ins

Medium

MCQ

[OK] (A) An iterator of (index, value) tuples

(B) The length of an iterable

(C) A numbered string

(D) A range object

Correct Answer: (A)
Explanation: enumerate(iterable, start=0) yields (index, element) tuples, useful for getting both index and value in a loop.

Q111. What is the purpose of the __slots__ attribute in a class?

Built-ins

Hard

MCQ

[OK] (A) Restricts instance attributes to a fixed set, saving memory

(B) Defines time slots for methods

(C) Creates database slots

(D) Limits method count

Correct Answer: (A)
Explanation: __slots__ replaces the per-instance __dict__ with a fixed set of attribute slots, reducing memory overhead.

Q112. What does abs() return?

Built-ins

Easy

MCQ

[OK] (A) The absolute value of a number

(B) The abstract class

(C) The first element

(D) A boolean

Correct Answer: (A)
Explanation: abs(x) returns the absolute value of x. For complex numbers, it returns the magnitude.

Q113. What does the any() function do?

Built-ins

Medium

MCQ

[OK] (A) Returns True if any element in the iterable is truthy

(B) Returns True if all elements are truthy

(C) Picks a random element

(D) Checks if the iterable is empty

Correct Answer: (A)
Explanation: any(iterable) returns True if at least one element is truthy. It short-circuits on the first True value.

Q114. What does the id() function return?

Built-ins

Hard

MCQ

[OK] (A) The identity (memory address in CPython) of an object

(B) A unique string identifier

(C) The object's hash

(D) The object's type

Correct Answer: (A)
Explanation: id(obj) returns a unique integer identifier for the object. In CPython, this is the memory address.

Q115. What is collections.defaultdict?

Collections

Medium

MCQ

[OK] (A) A dict subclass that provides default values for missing keys

(B) A read-only dictionary

(C) A sorted dictionary

(D) A dict that limits size

Correct Answer: (A)
Explanation: defaultdict takes a factory function (e.g., list, int) and uses it to create default values for missing keys.

Q116. What is collections.Counter used for?

Collections

Medium

MCQ

[OK] (A) Counting hashable objects and returning a dict of counts

(B) Counting loop iterations

(C) A thread-safe counter

(D) Counting lines in a file

Correct Answer: (A)

Explanation: Counter is a dict subclass for counting hashable objects. Counter('abcabc') gives Counter({'a':2,'b':2,'c':2}).

Q117. What is collections.OrderedDict in Python 3.7+?

Collections

Hard

MCQ

[OK] (A) A dict subclass that remembers order and supports reordering operations

(B) Identical to dict with no extra features

(C) A dict sorted by keys

(D) A deprecated class

Correct Answer: (A)

Explanation: While regular dicts maintain insertion order since 3.7, OrderedDict still offers move_to_end() and equality considers order.

Q118. What is a namedtuple?

Collections

Easy

MCQ

[OK] (A) A tuple subclass with named fields accessible by attribute

(B) A dictionary with ordered keys

(C) A named variable

(D) A tuple of names

Correct Answer: (A)
Explanation: namedtuple creates tuple subclasses with named fields, e.g., Point = namedtuple('Point', ['x', 'y']).

Q119. What is collections.ChainMap?

Collections

Hard

MCQ

[OK] (A) A class that groups multiple dicts into a single view for lookups

(B) A mapping with chained keys

(C) A hash chain implementation

(D) A blockchain-style dictionary

Correct Answer: (A)
Explanation: ChainMap groups multiple dicts and searches them in order during lookup. It is useful for scoped configurations.

Q120. What is the LEGB rule in Python?

Scope

Easy

MCQ

[OK] (A) Local, Enclosing, Global, Built-in - the order of name lookup

(B) Load, Execute, Generate, Build

(C) List, Element, Group, Block

(D) Lexical, Environmental, General, Base

Correct Answer: (A)
Explanation: LEGB describes Python's variable lookup order: Local scope, Enclosing function scope, Global scope, Built-in scope.

Q121. What does the global keyword do?

Scope

Medium

MCQ

[OK] (A) Declares that a variable inside a function refers to the global scope

(B) Creates a global function

(C) Exports a variable to other modules

(D) Makes a variable constant

Correct Answer: (A)
Explanation: The global keyword inside a function indicates that the variable refers to the module-level (global) variable.

Q122. What does the nonlocal keyword do?

Scope

Medium

MCQ

[OK] (A) Refers to a variable in the nearest enclosing scope (not global)

(B) Creates a variable outside any function

(C) Deletes a local variable

(D) Makes a variable thread-local

Correct Answer: (A)
Explanation: nonlocal binds a variable to the nearest enclosing function scope, enabling closures to modify outer variables.

Q123. What happens if you assign to a variable name inside a function without global/nonlocal?

Scope

Hard

MCQ

[OK] (A) Python treats it as a local variable for the entire function

(B) It modifies the global variable

(C) It raises NameError

(D) It creates both local and global variables

Correct Answer: (A)
Explanation: Any assignment in a function makes the name local to the entire function. Referencing before assignment raises UnboundLocalError.

Q124. What is a closure in Python?

Closures

Medium

MCQ

[OK] (A) A nested function that remembers variables from its enclosing scope

(B) A function that closes files

(C) A class destructor

(D) A locked function

Correct Answer: (A)
Explanation: A closure is an inner function that captures and remembers free variables from the enclosing function even after it returns.

Q125. When is a closure created in Python?

Closures

Medium

MCQ

[OK] (A) When a nested function references a variable from its enclosing scope

(B) When a function is imported

(C) When a class is instantiated

(D) When a module is loaded

Correct Answer: (A)
Explanation: A closure is formed when a nested function references free variables from the enclosing function scope.

Closures

Hard

MCQ

Q126. What attribute of a function object stores its closure variables?

[OK] (A) __closure__

(B) __vars__

(C) __scope__

(D) __env__

Correct Answer: (A)
Explanation: func.__closure__ is a tuple of cell objects containing the closed-over variables.

Closures

Hard

MCQ

Q127. What is the late binding issue with closures in loops?

[OK] (A) Closures capture variables by reference, so loop variables share the last value

(B) Closures cannot be used in loops

(C) Loop closures are always eager

(D) It only affects while loops

Correct Answer: (A)
Explanation: Closures capture the variable itself (not its value). In a loop, all closures share the same variable, which holds the final iteration value.

Advanced Python

Q128. What is a metaclass in Python?

Hard

MCQ

[OK] (A) A class whose instances are classes (the class of a class)

(B) A deprecated feature

(C) A mixin class

(D) An abstract class

Correct Answer: (A)
Explanation: Metaclasses define how classes behave. type is the default metaclass. Custom metaclasses inherit from type.

Advanced Python

Q129. What is monkey patching in Python?

Hard

MCQ

[OK] (A) Dynamically modifying a class or module at runtime

(B) A testing framework

(C) A code obfuscation technique

(D) A packaging tool

Correct Answer: (A)
Explanation: Monkey patching replaces attributes/methods of objects at runtime. It is powerful but can make code harder to maintain.

Advanced Python

Q130. What is the purpose of __all__ in a module?

Medium

MCQ

[OK] (A) Defines the public interface for 'from module import *'

(B) Lists all classes in the module

(C) Imports all submodules

(D) Prevents any imports

Correct Answer: (A)
*Explanation: __all__ is a list of names that should be exported when 'from module import *' is used.*

Advanced Python

Q131. What does the weakref module provide?

Hard

MCQ

[OK] (A) References to objects that do not prevent garbage collection

(B) Weak encryption

(C) Thread-safe references

(D) Compressed references

Correct Answer: (A)
Explanation: weakref creates references that do not increase the reference count, allowing the referenced object to be garbage collected.

Advanced Python

Medium

MCQ

Q132. What is the purpose of the __enter__ and __exit__ methods?

[OK] (A) They implement the context manager protocol for use with 'with' statements

(B) They control object creation and deletion

(C) They manage imports

(D) They handle exceptions only

Correct Answer: (A)
Explanation: __enter__ is called when entering a with block, __exit__ when leaving. Together they form the context manager protocol.

Data Types

Hard

MCQ

Q133. What is the difference between is and == in Python?

[OK] (A) 'is' checks identity (same object); '==' checks equality (same value)

(B) 'is' checks value; '==' checks identity

(C) They are identical

(D) 'is' only works with strings

Correct Answer: (A)
Explanation: 'is' compares object identity (id). '==' compares values using __eq__. Two objects can be equal but not identical.

Q134. What is the time complexity of string concatenation in a loop using +=?

Strings

Hard

MCQ

[OK] (A) O(n^2) in the worst case because strings are immutable

(B) O(n)

(C) O(1)

(D) O(n log n)

Correct Answer: (A)

Explanation: Each += creates a new string and copies existing content. Over n concatenations, this results in O(n^2) total work.

Q135. What is the difference between a shallow copy and a deep copy of a list?

Lists

Hard

MCQ

[OK] (A) Shallow copy copies references to nested objects; deep copy recursively copies all objects

(B) They are identical

(C) Shallow copy is slower

(D) Deep copy only works on flat lists

Correct Answer: (A)

Explanation: copy.copy() creates a shallow copy (nested objects are shared). copy.deepcopy() recursively copies all nested objects.

Dicts

Q136. What does the | operator do with dictionaries in Python 3.9+?

Hard

MCQ

[OK] (A) Merges two dictionaries, with the right one taking precedence

(B) Performs bitwise OR on values

(C) Checks for key overlap

(D) Creates a set from dict keys

Correct Answer: (A)
Explanation: In Python 3.9+, d1 | d2 merges dictionaries. If keys overlap, values from d2 take precedence.

Functions

Q137. What are positional-only parameters in Python 3.8+?

Hard

MCQ

[OK] (A) Parameters before / in a function signature that must be passed positionally

(B) Parameters that must be keywords

(C) Parameters with default values

(D) Parameters in lambda functions

Correct Answer: (A)
Explanation: Parameters before / in def f(a, b, /, c) must be passed as positional arguments, not keyword arguments.

Q138. What is __new__ used for in Python?

OOP

Hard

MCQ

[OK] (A) Creating a new instance before __init__ is called

(B) Renewing an existing instance

(C) Creating a new module

(D) Allocating memory manually

Correct Answer: (A)
Explanation: __new__ is a static method called before __init__. It actually creates the instance, while __init__ initializes it.

Q139. Can a class be used as a decorator in Python?

Decorators

Medium

MCQ

[OK] (A) Yes, by implementing __call__

(B) No, only functions can be decorators

(C) Only with @classmethod

(D) Only abstract classes

Correct Answer: (A)
Explanation: Any callable can be a decorator. A class with __call__ method can decorate functions by returning a callable instance.

Generators

Medium

MCQ

Q140. What is the difference between return and yield in a function?

[OK] (A) return ends the function; yield pauses it and can resume

(B) They are identical

(C) yield ends the function

(D) return pauses the function

Correct Answer: (A)
Explanation: return terminates the function and sends a value back. yield suspends the function, preserving state for resumption.

Comprehensions

Hard

MCQ

Q141. What is the set comprehension syntax in Python?

[OK] (A) {expr for item in iterable}

(B) [expr for item in iterable]

(C) (expr for item in iterable)

(D)

Correct Answer: (A)
*Explanation: Set comprehensions use curly braces like {x**2 for x in range(5)}, creating a set of unique values.*

Exception Handling

Q142. What is the base class for all built-in exceptions in Python?

Medium

MCQ

[OK] (A) BaseException

(B) Exception

(C) Error

(D) Throwable

Correct Answer: (A)
Explanation: BaseException is the root class. Exception is a subclass that most user-catchable exceptions derive from.

File I/O

Q143. What is memory-mapped file I/O in Python?

Hard

MCQ

[OK] (A) Using mmap to map a file into memory for efficient random access

(B) Reading entire files into memory

(C) Using virtual memory for files

(D) Caching files in RAM

Correct Answer: (A)
Explanation: The mmap module maps files into memory, allowing array-like access that is efficient for large files with random access patterns.

Modules

Q144. What is a namespace package in Python 3?

Hard

MCQ

[OK] (A) A package without __init__.py that can span multiple directories

(B) A package with named exports

(C) A package in a namespace

(D) A virtual package

Correct Answer: (A)
Explanation: Namespace packages (PEP 420) allow package portions to be split across multiple directories without __init__.py files.

Regular Expressions

Q145. What does re.compile() return?

Medium

MCQ

[OK] (A) A compiled regular expression pattern object

(B) A string

(C) A boolean

(D) A match object

Correct Answer: (A)
Explanation: re.compile(pattern) pre-compiles a regex for reuse, improving performance when the same pattern is used multiple times.

Q146. What does the reversed() function return?

Built-ins

Medium

MCQ

[OK] (A) A reverse iterator

(B) A reversed list

(C) A reversed string

(D) None

Correct Answer: (A)
Explanation: reversed() returns a reverse iterator. To get a list, wrap it: list(reversed(seq)).

Q147. What is collections.deque optimized for?

Collections

Medium

MCQ

[OK] (A) Fast appends and pops from both ends

(B) Fast random access

(C) Sorting

(D) Memory efficiency

Correct Answer: (A)
Explanation: deque (double-ended queue) provides O(1) append and pop from both ends, unlike lists which are O(n) for pop(0).

Scope

Q148. What is name mangling in Python?

Hard

MCQ

[OK] (A) Rewriting __attr to _ClassName__attr to avoid accidental overrides in subclasses

(B) Encrypting variable names

(C) Removing unused variables

(D) Renaming modules on import

Correct Answer: (A)
Explanation: Names starting with __ (double underscore) in a class are mangled to _ClassName__name to prevent accidental name clashes in inheritance.

Advanced Python

Q149. What is the Global Interpreter Lock's impact on multiprocessing?

Hard

MCQ

[OK] (A) No impact - multiprocessing uses separate processes, each with its own GIL

(B) It prevents multiprocessing entirely

(C) It makes multiprocessing slower

(D) It shares the GIL across processes

Correct Answer: (A)
Explanation: multiprocessing bypasses the GIL by using separate processes, each with its own Python interpreter and GIL.

Q150. What is the difference between a bytes object and a bytearray?

Data Types

Medium

MCQ

[OK] (A) bytes is immutable; bytearray is mutable

(B) They are identical

(C) bytes is mutable; bytearray is immutable

(D) bytearray is faster

Correct Answer: (A)

Explanation: bytes objects are immutable sequences of bytes. bytearray is the mutable counterpart, allowing in-place modification.

Q151. What is a recursive function?

Functions

Medium

MCQ

[OK] (A) A function that calls itself to solve a problem

(B) A function with multiple return statements

(C) A function that uses generators

(D) A function with no parameters

Correct Answer: (A)

Explanation: Recursive functions call themselves with modified arguments, typically with a base case to stop the recursion.

Q152. What is polymorphism in Python?

OOP

Medium

MCQ

[OK] (A) The ability of different classes to provide their own implementation of the same method

(B) Having multiple constructors

(C) Using global variables in classes

(D) Creating multiple instances

Correct Answer: (A)
Explanation: Polymorphism allows different classes to implement the same interface (method name) with different behaviors.

Q153. Which method checks if a string starts with a given prefix?

Strings

Medium

MCQ

(A) begins()

[OK] (B) startswith()

(C) prefix()

(D) has_start()

Correct Answer: (B)
Explanation: str.startswith(prefix) returns True if the string begins with the specified prefix.

Q154. What does list.pop() return?

Lists

Easy

MCQ

(A) The first element

[OK] (B) The last element and removes it

(C) None

(D) The list length

Correct Answer: (B)
Explanation: list.pop() removes and returns the last element. With an index argument, it removes and returns that element.

Q155. What does dict.pop(key, default) do when key is missing?

Dicts

Medium

MCQ

(A) Raises KeyError

[OK] (B) Returns the default value

(C) Returns None always

(D) Deletes the dictionary

Correct Answer: (B)
Explanation: dict.pop(key, default) returns and removes the value if key exists; otherwise returns default without raising an error.

Q156. What is the correct way to define a function with a default parameter?

Functions

Easy

MCQ

(A) def f(x = 10):

[OK] (B) def f(x=10):

(C) def f(x:10):

(D) def f(x->10):

Correct Answer: (B)
Explanation: Default parameters are defined with = in the function signature. Both x=10 and x = 10 work, but PEP 8 recommends no spaces.

Q157. What is an instance of a class?

OOP

Easy

MCQ

(A) The class definition itself

[OK] (B) A specific object created from the class

(C) A method of the class

(D) A class variable

Correct Answer: (B)
Explanation: An instance is a concrete object created by calling the class. Each instance has its own attribute values.

Exception Handling

Q158. Which of these correctly catches a ValueError?

Easy

MCQ

(A) try: ... catch ValueError:

[OK] (B) try: ... except ValueError:

(C) try: ... handle ValueError:

(D) try: ... rescue ValueError:

Correct Answer: (B)
Explanation: Python uses 'except' keyword to catch exceptions: try: ... except ValueError: ...

Built-ins

Q159. What does round(3.7) return?

Easy

MCQ

(A) 3

[OK] (B) 4

(C) 3.7

(D) 3

Correct Answer: (B)
Explanation: round(3.7) rounds to the nearest integer, returning 4. round() uses banker's rounding for .5 values.

Q160. Which of the following creates an empty tuple?

Tuples

Easy

MCQ

(A) (,)

[OK] (B) ()

(C) []

(D) {}

Correct Answer: (B)
Explanation: Empty parentheses () create an empty tuple. [] creates a list, {} creates a dict.

Q161. If class B(A) does not define __init__, what happens when B() is called?

Inheritance

Medium

MCQ

(A) An error is raised

[OK] (B) A's __init__ is called

(C) No initialization occurs

(D) A default __init__ is generated

Correct Answer: (B)
Explanation: Python follows the MRO. If B does not define __init__, it inherits and uses A's __init__.

Iterators

Hard

MCQ

Q162. What does itertools.chain() do?

(A) Links iterators permanently

[OK] (B) Chains multiple iterables into one sequential iterator

(C) Creates a linked list

(D) Chains function calls

Correct Answer: (B)
*Explanation: itertools.chain(*iterables) concatenates iterables, yielding elements from each in sequence.*

Data Types

Easy

MCQ

Q163. Which of the following is a mutable data type?

(A) int

(B) str

[OK] (C) list

(D) tuple

Correct Answer: (C)
Explanation: Lists are mutable in Python. Integers, strings, and tuples are all immutable.

Strings

Q164. What is the result of 'hello' + ' ' + 'world'?

Easy

MCQ

(A) helloworld

(B) hello world

[OK] (C) hello world

(D) Error

Correct Answer: (C)
Explanation: String concatenation with + joins the strings as-is. The space in the middle produces 'hello world'.

Sets

Q165. How do you create an empty set?

Easy

MCQ

(A) {}

(B) set[]

[OK] (C) set()

(D) empty_set()

Correct Answer: (C)
Explanation: set() creates an empty set. {} creates an empty dictionary, not a set.

Q166. What does the return keyword do in a function?

Functions

Easy

MCQ

(A) Prints a value

(B) Pauses execution

[OK] (C) Exits the function and optionally sends a value back

(D) Loops through values

Correct Answer: (C)
Explanation: return exits the function immediately and optionally returns a value to the caller.

Q167. What mode appends to a file without truncating it?

File I/O

Medium

MCQ

(A) 'w'

(B) 'r'

[OK] (C) 'a'

(D) 'x'

Correct Answer: (C)
Explanation: 'a' (append) mode opens the file for writing at the end. Unlike 'w', it does not truncate existing content.

Regular Expressions

Q168. What does the + quantifier mean in regex?

Easy

MCQ

(A) Zero or more occurrences

(B) Exactly one occurrence

[OK] (C) One or more occurrences

(D) Zero or one occurrence

Correct Answer: (C)
*Explanation: + matches one or more occurrences of the preceding element. * matches zero or more, ? matches zero or one.*

Collections

Q169. What method returns the n most common elements from a Counter?

Easy

MCQ

(A) top(n)

(B) frequent(n)

[OK] (C) most_common(n)

(D) best(n)

Correct Answer: (C)
Explanation: Counter.most_common(n) returns a list of the n most common elements and their counts.

Advanced Python

Medium

MCQ

Q170. What tool is commonly used for type checking in Python?

(A) pylint

(B) black

[OK] (C) mypy

(D) flake8

Correct Answer: (C)
Explanation: mypy is the most widely used static type checker for Python, using type annotations (PEP 484) to find type errors.

Closures

Hard

MCQ

Q171. How do you fix the late binding issue in a closure inside a loop?

(A) Use global variables

(B) Use a class instead

[OK] (C) Use a default argument to capture the current value

(D) Use threading

Correct Answer: (C)
Explanation: Using a default argument (lambda x=x: x) captures the current value at each iteration, avoiding late binding.

Data Types

Medium

MCQ

Q172. Which of the following evaluates to True in Python?

(A) bool(0)

(B) bool('')

(C) bool([])

[OK] (D) bool(' ')

Correct Answer: (D)
Explanation: A string containing a space is non-empty and therefore truthy. 0, empty string, and empty list are all falsy.

Lists

Medium

MCQ

Q173. Which of the following creates a deep copy of a list?

(A) b = a

(B) b = a[:]

(C) b = list(a)

[OK] (D) b = copy.deepcopy(a)

Correct Answer: (D)
Explanation: copy.deepcopy() recursively copies all nested objects. The others create shallow copies or mere references.

Q174. Which method removes all items from a dictionary?

Dicts

Easy

MCQ

(A) delete()

(B) remove()

(C) pop()

[OK] (D) clear()

Correct Answer: (D)
Explanation: dict.clear() removes all key-value pairs from the dictionary. pop() removes a single key.

Q175. What denotes a 'private' attribute in Python by convention?

OOP

Medium

MCQ

(A) private keyword

(B) protected keyword

(C) Single underscore prefix _

[OK] (D) Double underscore prefix __

Correct Answer: (D)
Explanation: Double underscore prefix triggers name mangling (_ClassName__attr), providing stronger access protection by convention.

Q176. Which of the following is NOT a valid way to import in Python?

Modules

Medium

MCQ

(A) import os

(B) from os import path

(C) from os import path as p

[OK] (D) import os.path as os

Correct Answer: (D)
Explanation: While syntactically valid, aliasing os.path as 'os' shadows the os module and is a problematic import practice.

Q177. What does print() do by default at the end of output?

Built-ins

Easy

MCQ

(A) Adds a tab

(B) Adds a space

(C) Adds nothing

[OK] (D) Adds a newline

Correct Answer: (D)
Explanation: print() adds a newline character at the end by default. Use end='' to suppress it.

Q178. In def f(a, b, *, c), what is c?

Functions

Medium

MCQ

(A) A positional argument

(B) A default argument

(C) A variable-length argument

[OK] (D) A keyword-only argument

Correct Answer: (D)
*Explanation: Parameters after * must be passed as keyword arguments. c must be called as f(1, 2, c=3).*

Q179. What scope does a variable defined at the top level of a module have?

Scope

Medium

MCQ

(A) Local

(B) Enclosing

(C) Built-in

[OK] (D) Global

Correct Answer: (D)
Explanation: Variables defined at module level have global scope. They are accessible throughout the module.

Generators

Q180. What method is used to throw an exception into a generator?

Hard

MCQ

(A) raise()

(B) error()

(C) stop()

[OK] (D) throw()

Correct Answer: (D)
Explanation: gen.throw(ExcType) throws an exception at the point where the generator is paused (at the yield expression).

Advanced Python

Q181. Which Python module provides support for data classes?

Hard

MCQ

(A) typing

(B) collections

(C) abc

[OK] (D) dataclasses

Correct Answer: (D)
Explanation: The dataclasses module (Python 3.7+) provides @dataclass decorator for auto-generating __init__, __repr__, etc.

Strings

Medium

MCQ

Q182. What does 'hello'.center(11, '*') return?

[OK] (A) '*hello***'**

(B) '*hello*****'

(C) '*****hello*'

(D) 'hello******'

Correct Answer: (A)
Explanation: center(width, fillchar) centers the string in a field of given width, padding with the fill character on both sides.

Tuples

Medium

MCQ

Q183. What is the result of (1, 2) + (3, 4)?

[OK] (A) (1, 2, 3, 4)

(B) (4, 6)

(C) Error

(D) ((1,2),(3,4))

Correct Answer: (A)
Explanation: The + operator concatenates tuples, creating a new tuple with all elements from both.

Q184. What is a class decorator in Python?

Decorators

Hard

MCQ

[OK] (A) A decorator applied to a class definition to modify or augment it

(B) A decorator defined inside a class

(C) A class method decorator

(D) A decorator that returns a class

Correct Answer: (A)
Explanation: Class decorators are applied to class definitions using @decorator syntax. @dataclass is a common example.

Q185. What does itertools.islice(iterable, 5) do?

Iterators

Medium

MCQ

(A) Returns the last 5 elements

(B) Returns elements at index 5

(C) Takes every 5th element

[OK] (D) Returns the first 5 elements

Correct Answer: (D)
Explanation: itertools.islice(iterable, 5) returns an iterator over the first 5 elements, similar to iterable[:5] but for any iterator.

File I/O

Easy

MCQ

Q186. What method reads all lines of a file into a list?

(A) read()

(B) readline()

(C) getlines()

[OK] (D) readlines()

Correct Answer: (D)
Explanation: readlines() reads all lines and returns them as a list. read() returns the entire content as a string.

Data Types

Easy

Output

Q187. What is the output of the following code?

```
print(type(42))
```

[OK] (A)

(B)

(C) int

(D) 42

Correct Answer: (A)
Explanation: type() returns the type of an object. For 42, it returns .

Q188. What will this code print?

Data Types

Easy

Output

```
x = 10
y = 3
print(x // y)
```

[OK] (A) 3

(B) 3.33

(C) 4

(D) 10

Correct Answer: (A)
Explanation: // is floor division. 10 // 3 = 3 (rounds down to the nearest integer).

Q189. What is the output?

Data Types

Medium

Output

```
print(0.1 + 0.2 == 0.3)
```

[OK] (A) FALSE

(B) TRUE

(C) 0.3

(D) Error

Correct Answer: (A)
Explanation: Due to floating-point representation, 0.1 + 0.2 is 0.30000000000000004, which is not equal to 0.3.

Q190. What does this code output?

Strings

Easy

Output

```
s = 'Python'
print(s[0:3])
```

[OK] (A) Pyt

(B) Pyth

(C) Py

(D) Python

Correct Answer: (A)
Explanation: Slicing s[0:3] extracts characters at indices 0, 1, 2, giving 'Pyt'.

Q191. What is printed?

Strings

Medium

Output

```
print('abc' * 3)
```

[OK] (A) abcabcabc

(B) abc3

(C) abc abc abc

(D) Error

Correct Answer: (A)
*Explanation: The * operator repeats a string. 'abc' * 3 produces 'abcabcabc'.*

Q192. What is the output?

Strings

Hard

Output

```
s = 'hello'
print(s.replace('l', 'L', 1))
```

[OK] (A) heLlo

(B) heLLo

(C) HeLlo

(D) hello

Correct Answer: (A)
Explanation: replace() with count=1 replaces only the first occurrence. The first 'l' becomes 'L'.

Q193. What does this code print?

Lists

Easy

Output

```
lst = [1, 2, 3, 4, 5]
print(lst[-1])
```

[OK] (A) 5

(B) 1

(C) 4

(D) Error

Correct Answer: (A)
Explanation: Negative indexing starts from the end. lst[-1] is the last element, which is 5.

Q194. What is the output?

Lists

Medium

Output

```
a = [1, 2, 3]
b = a[:]
b.append(4)
print(a)
```

[OK] (A) [1, 2, 3]

(B) [1, 2, 3, 4]

(C) [4, 1, 2, 3]

(D) Error

Correct Answer: (A)
Explanation: a[:] creates a shallow copy. Modifying b does not affect a, so a remains [1, 2, 3].

Q195. What is printed?

Lists

Hard

Output

```
x = [[]] * 3
x[0].append(1)
print(x)
```

[OK] (A) [[1], [1], [1]]

(B) [[1], [], []]

(C) [1, [], []]

(D) Error

Correct Answer: (A)
*Explanation: [[]] * 3 creates three references to the SAME inner list. Appending to one affects all three.*

Q196. What does this code output?

Dicts

Easy

Output

```
d = {'a': 1, 'b': 2, 'c': 3}
print(len(d))
```

[OK] (A) 3

(B) 6

(C) 2

(D) Error

Correct Answer: (A)
Explanation: len() on a dictionary returns the number of key-value pairs. This dictionary has 3 entries.

Q197. What is printed?

Dicts

Medium

Output

```
d = {'x': 1, 'y': 2}
print(d.get('z', 0))
```

[OK] (A) 0

(B) None

(C) KeyError

(D) z

Correct Answer: (A)
Explanation: dict.get(key, default) returns default when the key is missing. 'z' is not in d, so 0 is returned.

Q198. What is the output?

Sets

Easy

Output

```
s = {1, 2, 2, 3, 3, 3}
print(len(s))
```

[OK] (A) 3

(B) 6

(C) 1

(D) Error

Correct Answer: (A)
Explanation: Sets contain unique elements only. Duplicates are removed, leaving {1, 2, 3} with length 3.

Q199. What does this print?

Sets

Medium

Output

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)
```

[OK] (A) {1, 2, 3, 4}

(B) {2, 3}

(C) {1, 4}

(D) Error

Correct Answer: (A)
Explanation: The | operator computes the union of two sets, combining all unique elements from both.

Q200. What is the output?

Tuples

Easy

Output

```
t = (1, 2, 3)
print(t[1])
```

[OK] (A) 2

(B) 1

(C) 3

(D) (1, 2)

Correct Answer: (A)
Explanation: Tuple indexing is zero-based. t[1] returns the second element, which is 2.

Q201. What does this code print?

Tuples

Medium

Output

```
a, *b, c = (1, 2, 3, 4, 5)
print(b)
```

[OK] (A) [2, 3, 4]

(B) (2, 3, 4)

(C) [1, 2, 3]

(D) Error

Correct Answer: (A)
*Explanation: Extended unpacking with *b captures the middle elements as a list: a=1, b=[2,3,4], c=5.*

Q202. What is the output?

Functions

Easy

Output

```
def add(a, b):
return a + b
print(add(3, 4))
```

[OK] (A) 7

(B) 34

(C) 12

(D) None

Correct Answer: (A)
Explanation: The function adds 3 + 4 = 7 and returns it. print() displays 7.

Q203. What does this code print?

Functions

Medium

Output

```
def f(x=[]):
x.append(1)
return x
print(f())
print(f())
```

[OK] (A) [1]\n[1, 1]

(B) [1]\n[1]

(C) [1, 1]\n[1, 1]

(D) Error

Correct Answer: (A)
Explanation: The mutable default list is shared across calls. First call appends 1, second call appends another 1.

Q204. What is printed?

Functions

Hard

Output

```
def outer():
x = 10
def inner():
nonlocal x
x += 5
return x
return inner()
print(outer())
```

[OK] (A) 15

(B) 10

(C) 5

(D) Error

Correct Answer: (A)
Explanation: nonlocal x refers to x in outer. inner() modifies x from 10 to 15 and returns it.

Q205. What does this code output?

OOP

Medium

Output

```
class Dog:
species = 'Canis'
def __init__(self, name):
self.name = name
d = Dog('Rex')
print(d.species, d.name)
```

[OK] (A) Canis Rex

(B) Dog Rex

(C) Rex Canis

(D) Error

Correct Answer: (A)

Explanation: species is a class attribute ('Canis'). name is an instance attribute ('Rex'). Both are accessible via the instance.

Q206. What is printed?

OOP

Hard

Output

```
class A:
def greet(self):
return 'Hello from A'
class B(A):
def greet(self):
return 'Hello from B'
class C(A):
pass
print(C().greet())
```

[OK] (A) Hello from A

(B) Hello from B

(C) None

(D) Error

Correct Answer: (A)
Explanation: C inherits from A (not B). C does not override greet(), so A's greet() is called.

Q207. What is the output?

Inheritance

Medium

Output

```
class Base:
def __init__(self):
self.x = 1
class Child(Base):
def __init__(self):
super().__init__()
self.y = 2
c = Child()
print(c.x, c.y)
```

[OK] (A) 1 2

(B) 2 1

(C) 1 1

(D) Error

Correct Answer: (A)
Explanation: super().__init__() calls Base.__init__ which sets x=1. Then Child sets y=2.

Q208. What does this code print?

Decorators

Medium

Output

```
def double(func):
def wrapper(*args):
return func(*args) * 2
return wrapper

@double
def add(a, b):
return a + b

print(add(3, 4))
```

[OK] (A) 14

(B) 7

(C) 34

(D) Error

Correct Answer: (A)
Explanation: The decorator doubles the return value. add(3,4) returns 7, multiplied by 2 gives 14.

Q209. What is printed?

Generators

Medium

Output

```
def gen():
yield 1
yield 2
yield 3
g = gen()
print(next(g))
print(next(g))
```

[OK] (A) 1\n2

(B) 1\n1

(C) 3\n3

(D) Error

Correct Answer: (A)

Explanation: Each next() call resumes the generator to the next yield. First yields 1, second yields 2.

Q210. What is the output?

Generators

Hard

Output

```
def fib():
a, b = 0, 1
while True:
yield a
a, b = b, a + b
f = fib()
print([next(f) for _ in range(6)])
```

[OK] (A) [0, 1, 1, 2, 3, 5]

(B) [1, 1, 2, 3, 5, 8]

(C) [0, 1, 2, 3, 5, 8]

(D) Error

Correct Answer: (A)
Explanation: The generator yields Fibonacci numbers starting from 0. The first 6 values are 0, 1, 1, 2, 3, 5.

Q211. What does this code output?

Comprehensions

Easy

Output

```
print([x**2 for x in range(5)])
```

[OK] (A) [0, 1, 4, 9, 16]

(B) [1, 4, 9, 16, 25]

(C) [0, 2, 4, 6, 8]

(D) Error

Correct Answer: (A)
Explanation: range(5) gives 0,1,2,3,4. Squaring each: 0,1,4,9,16.

Q212. What is printed?

Comprehensions

Medium

Output

```
print([x for x in range(10) if x % 2 == 0])
```

[OK] (A) [0, 2, 4, 6, 8]

(B) [1, 3, 5, 7, 9]

(C) [2, 4, 6, 8, 10]

(D) [0, 1, 2, 3, 4]

Correct Answer: (A)
Explanation: The comprehension filters even numbers from 0-9. Even numbers in range(10) are 0, 2, 4, 6, 8.

Q213. What is the output?

Exception Handling

Easy

Output

```
try:
x = 1 / 0
except ZeroDivisionError:
print('caught')
```

[OK] (A) caught

(B) ZeroDivisionError

(C) 0

(D) Error

Correct Answer: (A)
Explanation: Division by zero raises ZeroDivisionError, which is caught by the except clause, printing 'caught'.

Exception Handling

Q214. What does this code print?

Medium

Output

```
try:
x = int('abc')
except ValueError:
print('bad value')
finally:
print('done')
```

[OK] (A) bad value\ndone

(B) done

(C) bad value

(D) Error

Correct Answer: (A)
Explanation: int('abc') raises ValueError which is caught. Then finally executes regardless, printing both messages.

Q215. What is printed?

Scope

Medium

Output

```
x = 'global'
def f():
x = 'local'
print(x)
f()
print(x)
```

[OK] (A) local\nglobal

(B) global\nglobal

(C) local\nlocal

(D) Error

Correct Answer: (A)
Explanation: Inside f(), x is a local variable ('local'). Outside, x is the global variable ('global').

Q216. What is the output?

Scope

Hard

Output

```
x = 1
def f():
global x
x = 2
f()
print(x)
```

[OK] (A) 2

(B) 1

(C) None

(D) Error

Correct Answer: (A)
Explanation: The global keyword makes x refer to the global variable. f() changes it from 1 to 2.

Q217. What does this code print?

Closures

Medium

Output

```
def make_adder(n):
def adder(x):
return x + n
return adder
add5 = make_adder(5)
print(add5(10))
```

[OK] (A) 15

(B) 10

(C) 5

(D) Error

Correct Answer: (A)
Explanation: make_adder(5) returns a closure that adds 5 to its argument. add5(10) returns 10 + 5 = 15.

Q218. What is the output?

Closures

Hard

Output

```
funcs = []
for i in range(3):
funcs.append(lambda: i)
print([f() for f in funcs])
```

[OK] (A) [2, 2, 2]

(B) [0, 1, 2]

(C) [0, 0, 0]

(D) Error

Correct Answer: (A)
Explanation: Late binding: all lambdas reference the same variable i, which is 2 after the loop ends. All return 2.

Q219. What is the output?

Built-ins

Easy

Output

```
print(max(3, 1, 4, 1, 5, 9))
```

[OK] (A) 9

(B) 1

(C) 5

(D) 3

Correct Answer: (A)
Explanation: max() returns the largest argument. Among 3, 1, 4, 1, 5, 9, the maximum is 9.

Q220. What does this print?

Built-ins

Medium

Output

```
print(list(zip([1,2,3], ['a','b','c'])))
```

[OK] (A) [(1, 'a'), (2, 'b'), (3, 'c')]

(B) [[1,'a'], [2,'b'], [3,'c']]

(C) {1:'a', 2:'b', 3:'c'}

(D) Error

Correct Answer: (A)
Explanation: zip pairs elements from both iterables into tuples. list() converts the zip object to a list.

Q221. What is the output?

Collections

Medium

Output

```
from collections import Counter
c = Counter('abracadabra')
print(c.most_common(2))
```

[OK] (A) [('a', 5), ('b', 2)]

(B) [('a', 5), ('r', 2)]

(C) {'a': 5, 'b': 2}

(D) [5, 2]

Correct Answer: (A)
Explanation: Counter counts characters. 'a' appears 5 times, 'b' appears 2 times. most_common(2) returns the top 2.

Q222. What is printed?

Collections

Hard

Output

```
from collections import defaultdict
dd = defaultdict(list)
dd['key'].append(1)
dd['key'].append(2)
print(dd['key'])
```

[OK] (A) [1, 2]

(B) 1

(C) [1]

(D) Error

Correct Answer: (A)
Explanation: defaultdict(list) creates an empty list for missing keys. We append 1 and 2 to dd['key'], giving [1, 2].

Q223. What is the output?

Iterators

Easy

Output

```
it = iter([10, 20, 30])
print(next(it))
print(next(it))
```

[OK] (A) 10\n20

(B) 10\n10

(C) 30\n30

(D) Error

Correct Answer: (A)
Explanation: iter() creates an iterator. Each next() call returns the next element: first 10, then 20.

Q224. What is printed?

Iterators

Hard

Output

```
class Count:
def __init__(self, n):
self.n = n
self.i = 0
def __iter__(self):
return self
def __next__(self):
if self.i >= self.n:
raise StopIteration
self.i += 1
return self.i
print(list(Count(3)))
```

[OK] (A) [1, 2, 3]

(B) [0, 1, 2]

(C) [0, 1, 2, 3]

(D) Error

Correct Answer: (A)

Explanation: The iterator increments self.i before returning, yielding 1, 2, 3. StopIteration fires when i >= 3.

Q225. What is the output?

Regular Expressions

Medium

Output

```
import re
print(re.findall(r'\d+', 'abc123def456'))
```

[OK] (A) ['123', '456']

(B) ['abc', 'def']

(C) ['1', '2', '3', '4', '5', '6']

(D) Error

Correct Answer: (A)
Explanation: \d+ matches one or more digits. findall returns all matches: '123' and '456'.

Q226. What does this code print?

Regular Expressions

Hard

Output

```
import re
m = re.match(r'(\w+)@(\w+)', 'user@host')
print(m.groups())
```

[OK] (A) ('user', 'host')

(B) 'user@host'

(C) ['user', 'host']

(D) None

Correct Answer: (A)
Explanation: Two capturing groups (\w+) match 'user' and 'host'. groups() returns them as a tuple.

Advanced Python

Q227. What is the output?

Medium

Output

```
from dataclasses import dataclass

@dataclass
class Point:
x: int
y: int

p = Point(1, 2)
print(p)
```

[OK] (A) Point(x=1, y=2)

(B)

(C) (1, 2)

(D) Error

Correct Answer: (A)
Explanation: @dataclass auto-generates __repr__ that shows the class name and field values.

Q228. What is printed?

Advanced Python

Hard

Output

```
class Meta(type):
def __new__(mcs, name, bases, namespace):
namespace['version'] = 1
return super().__new__(mcs, name, bases, namespace)
class MyClass(metaclass=Meta):
pass
print(MyClass.version)
```

[OK] (A) 1

(B) None

(C) Error

(D) Meta

Correct Answer: (A)
Explanation: The metaclass __new__ adds 'version' to the namespace. MyClass gets version=1 as a class attribute.

Q229. What is the output?

Modules

Easy

Output

```
import math
print(math.floor(3.7))
```

[OK] (A) 3

(B) 4

(C) 3.7

(D) Error

Correct Answer: (A)
Explanation: math.floor() returns the largest integer less than or equal to the argument. floor(3.7) = 3.

Q230. What is the output?

Data Types

Hard

Output

```
a = 256
b = 256
print(a is b)
c = 257
d = 257
print(c is d)
```

[OK] (A) True\nFalse

(B) True\nTrue

(C) False\nFalse

(D) False\nTrue

Correct Answer: (A)
Explanation: CPython caches integers from -5 to 256. a and b share the same object (True). 257 creates separate objects (False).

Q231. What does this print?

Strings

Medium

Output

```
s = 'Hello, World!'
print(s.split(', '))
```

[OK] (A) ['Hello', 'World!']

(B) ['Hello,', 'World!']

(C) ['Hello, World!']

(D) Error

Correct Answer: (A)
Explanation: split(', ') splits on the comma-space delimiter, producing ['Hello', 'World!'].

Q232. What is the output?

Lists

Medium

Output

```
lst = [3, 1, 4, 1, 5]
lst.sort(reverse=True)
print(lst)
```

[OK] (A) [5, 4, 3, 1, 1]

(B) [1, 1, 3, 4, 5]

(C) [5, 3, 4, 1, 1]

(D) Error

Correct Answer: (A)
Explanation: sort(reverse=True) sorts in descending order. The sorted list is [5, 4, 3, 1, 1].

Q233. What is printed?

Dicts

Hard

Output

```
d = {'a': 1, 'b': 2}
d.update({'b': 3, 'c': 4})
print(d)
```

[OK] (A) {'a': 1, 'b': 3, 'c': 4}

(B) {'a': 1, 'b': 2, 'c': 4}

(C) {'b': 3, 'c': 4}

(D) Error

Correct Answer: (A)
Explanation: update() merges dictionaries. 'b' is updated to 3, 'c' is added with 4. 'a' remains unchanged.

Q234. What is the output?

Functions

Medium

Output

```
square = lambda x: x ** 2
print(square(5))
```

[OK] (A) 25

(B) 10

(C) 5

(D) Error

Correct Answer: (A)
*Explanation: The lambda creates a function that squares its argument. square(5) = 5 ** 2 = 25.*

Q235. What is printed?

Comprehensions

Hard

Output

```
matrix = [[1,2],[3,4],[5,6]]
flat = [x for row in matrix for x in row]
print(flat)
```

[OK] (A) [1, 2, 3, 4, 5, 6]

(B) [[1,2],[3,4],[5,6]]

(C) [1, 3, 5, 2, 4, 6]

(D) Error

Correct Answer: (A)
Explanation: The nested comprehension iterates rows then elements within each row, flattening the matrix.

Q236. What is the output?

Decorators

Hard

Output

```
def decorator(cls):
cls.decorated = True
return cls

@decorator
class MyClass:
pass

print(MyClass.decorated)
```

[OK] (A) TRUE

(B) FALSE

(C) None

(D) Error

Correct Answer: (A)
Explanation: The class decorator adds a 'decorated' attribute set to True on the class.

Exception Handling

Q237. What is the output?

Hard

Output

```
def divide(a, b):
try:
return a / b
except ZeroDivisionError:
return 'inf'
finally:
print('cleanup')
result = divide(10, 0)
print(result)
```

[OK] (A) cleanup\ninf

(B) inf\ncleanup

(C) inf

(D) cleanup

Correct Answer: (A)

Explanation: finally executes before the function returns. So 'cleanup' prints first, then 'inf' is returned and printed.

Q238. What is the output?

Generators

Medium

Output

```
def squares(n):
for i in range(n):
yield i ** 2
print(list(squares(4)))
```

[OK] (A) [0, 1, 4, 9]

(B) [1, 4, 9, 16]

(C) [0, 2, 4, 6]

(D) Error

Correct Answer: (A)
Explanation: The generator yields squares of 0,1,2,3: that is 0,1,4,9. list() collects all values.

Q239. What is the output?

Built-ins

Easy

Output

```
print(sorted([3, 1, 4, 1, 5, 9]))
```

[OK] (A) [1, 1, 3, 4, 5, 9]

(B) [9, 5, 4, 3, 1, 1]

(C) [3, 1, 4, 1, 5, 9]

(D) Error

Correct Answer: (A)
Explanation: sorted() returns a new list sorted in ascending order by default.

Q240. What is printed?

Built-ins

Medium

Output

```
print(list(enumerate(['a', 'b', 'c'], start=1)))
```

[OK] (A) [(1, 'a'), (2, 'b'), (3, 'c')]

(B) [(0, 'a'), (1, 'b'), (2, 'c')]

(C) [('a', 1), ('b', 2), ('c', 3)]

(D) Error

Correct Answer: (A)
Explanation: enumerate with start=1 begins counting from 1. Each element is paired with its index.

Q241. What is the output?

Advanced Python

Hard

Output

```
import sys
print(sys.getrecursionlimit() > 100)
```

[OK] (A) TRUE

(B) FALSE

(C) 100

(D) Error

Correct Answer: (A)
Explanation: Python's default recursion limit is usually 1000, which is greater than 100, so this prints True.

Q242. What does this print?

OOP

Medium

Output

```
class Animal:
def speak(self):
return 'generic sound'
class Cat(Animal):
def speak(self):
return 'meow'
print(Cat().speak())
```

[OK] (A) meow

(B) generic sound

(C) None

(D) Error

Correct Answer: (A)
Explanation: Cat overrides the speak method. Cat().speak() calls the Cat version, returning 'meow'.

Q243. What is printed?

Closures

Hard

Output

```
def counter():
count = 0
def increment():
nonlocal count
count += 1
return count
return increment
c = counter()
print(c(), c(), c())
```

[OK] (A) 1 2 3

(B) 1 1 1

(C) 0 1 2

(D) Error

Correct Answer: (A)
Explanation: The closure remembers count between calls. Each call increments and returns the updated value: 1, 2, 3.

Q244. What is the output?

Data Types

Medium

Output

```
print(bool(''), bool(' '), bool(0), bool(None))
```

[OK] (A) False True False False

(B) False False False False

(C) True True False False

(D) True True True True

Correct Answer: (A)
Explanation: Empty string is falsy, space string is truthy, 0 is falsy, None is falsy.

Q245. What is printed?

Lists

Hard

Output

```
a = [1, 2, 3]
print(a * 2)
```

[OK] (A) [1, 2, 3, 1, 2, 3]

(B) [2, 4, 6]

(C) [1, 2, 3, 2]

(D) Error

Correct Answer: (A)
*Explanation: The * operator on lists repeats the entire list. [1,2,3] * 2 creates [1,2,3,1,2,3].*

Q246. What is the output?

Strings

Hard

Output

```
name = 'Alice'
age = 30
print(f'{name!r} is {age}')
```

[OK] (A) 'Alice' is 30

(B) Alice is 30

(C) Alice!r is 30

(D) Error

Correct Answer: (A)
Explanation: !r applies repr() to the value. For strings, repr adds quotes, giving 'Alice' is 30.

Q247. What is printed?

Sets

Hard

Output

```
s = {1, 2, 3}
s.discard(4)
print(s)
```

[OK] (A) {1, 2, 3}

(B) Error

(C) {1, 2, 3, 4}

(D) set()

Correct Answer: (A)
Explanation: discard() removes an element if present but does NOT raise an error if the element is missing. The set is unchanged.

Q248. What is the output?

Tuples

Hard

Output

```
t = (1, [2, 3], 4)
t[1].append(5)
print(t)
```

[OK] (A) (1, [2, 3, 5], 4)

(B) TypeError

(C) (1, [2, 3], 4, 5)

(D) (1, [2, 3], 4)

Correct Answer: (A)
Explanation: The tuple is immutable, but its elements can be mutable. The list at t[1] can be modified in place.

Q249. What is the output?

Modules

Medium

Output

```
import os.path
print(os.path.join('home', 'user', 'file.txt'))
```

[OK] (A) home/user/file.txt

(B) home\user\file.txt

(C) /home/user/file.txt

(D) Error

Correct Answer: (A)
Explanation: os.path.join combines path components. On Unix-like systems, it uses forward slashes. The exact separator is OS-dependent.

Q250. What does this code print?

Inheritance

Hard

Output

```
class A:
x = 1
class B(A):
pass
class C(A):
pass
B.x = 2
print(A.x, B.x, C.x)
```

[OK] (A) 1 2 1

(B) 2 2 2

(C) 1 1 1

(D) 1 2 2

Correct Answer: (A)
Explanation: B.x = 2 creates a class attribute on B, shadowing A.x. C still inherits A.x = 1.

Q251. What is printed?

Dicts

Medium

Output

```
d = dict(zip(['a','b','c'], [1,2,3]))
print(d)
```

[OK] (A) {'a': 1, 'b': 2, 'c': 3}

(B) [('a',1),('b',2),('c',3)]

(C) {1:'a', 2:'b', 3:'c'}

(D) Error

Correct Answer: (A)
Explanation: zip pairs keys with values. dict() converts these pairs into a dictionary.

Q252. What is the output?

Functions

Hard

Output

```
def apply(func, *args):
return func(*args)
print(apply(pow, 2, 10))
```

[OK] (A) 1024

(B) 20

(C) 210

(D) Error

Correct Answer: (A)
Explanation: apply calls pow(2, 10) which computes 2^10 = 1024.

Q253. What does this print?

Comprehensions

Medium

Output

```
d = {k: v for k, v in zip('abc', [1,2,3])}
print(d)
```

[OK] (A) {'a': 1, 'b': 2, 'c': 3}

(B) {'abc': [1,2,3]}

(C) [('a',1),('b',2),('c',3)]

(D) Error

Correct Answer: (A)
Explanation: The dict comprehension creates key-value pairs from the zipped iterables.

Q254. What is printed?

Built-ins

Hard

Output

```
print(all([True, True, False]))
print(any([False, False, True]))
```

[OK] (A) False\nTrue

(B) True\nTrue

(C) False\nFalse

(D) True\nFalse

Correct Answer: (A)
Explanation: all() returns True only if ALL elements are truthy (False here). any() returns True if ANY element is truthy (True here).

Q255. What is the output?

Scope

Hard

Output

```
def f():
try:
print(x)
except NameError:
print('not found')
x = 10
f()
```

(A) not found

(B) 10

(C) None

[OK] (D) UnboundLocalError

Correct Answer: (D)
Explanation: Python determines x is local (due to assignment later). Accessing it before assignment raises UnboundLocalError, not caught by NameError.

Q256. What is the output?

File I/O

Medium

Output

```
import io
f = io.StringIO('hello world')
print(f.read(5))
```

[OK] (A) hello

(B) hello world

(C) h

(D) Error

Correct Answer: (A)
Explanation: StringIO is an in-memory text stream. read(5) reads the first 5 characters: 'hello'.

Advanced Python

Q257. What does this code print?

Medium

Output

```
x = [1, 2, 3]
y = [1, 2, 3]
print(x == y)
print(x is y)
```

[OK] (A) True\nFalse

(B) True\nTrue

(C) False\nFalse

(D) False\nTrue

Correct Answer: (A)
Explanation: == checks value equality (True). 'is' checks identity - x and y are separate objects (False).

Q258. What is the output?

Decorators

Hard

Output

```
def repeat(n):
def decorator(func):
def wrapper(*args):
return [func(*args) for _ in range(n)]
return wrapper
return decorator

@repeat(3)
def greet():
return 'hi'

print(greet())
```

[OK] (A) ['hi', 'hi', 'hi']

(B) 'hi'

(C) ('hi', 'hi', 'hi')

(D) Error

Correct Answer: (A)

Explanation: The decorator factory repeat(3) creates a decorator that calls greet() 3 times, collecting results in a list.

Q259. What is printed?

Collections

Medium

Output

```
from collections import namedtuple
Point = namedtuple('Point', ['x', 'y'])
p = Point(3, 4)
print(p.x, p[1])
```

[OK] (A) 3 4

(B) 3 3

(C) 4 3

(D) Error

Correct Answer: (A)
Explanation: Named tuples support both attribute access (p.x=3) and index access (p[1]=4).

Q260. What is the output?

Generators

Hard

Output

```
def gen():
yield from range(3)
yield from range(3, 6)
print(list(gen()))
```

[OK] (A) [0, 1, 2, 3, 4, 5]

(B) [0, 1, 2]

(C) [3, 4, 5]

(D) Error

Correct Answer: (A)
Explanation: 'yield from' delegates to each range. First yields 0,1,2 then 3,4,5, creating a combined sequence.

Q261. What is the output?

Regular Expressions

Hard

Output

```
import re
print(re.sub(r'\s+', '-', 'hello world python'))
```

[OK] (A) hello-world-python

(B) hello world python

(C) hello-world-python-

(D) #NAME?

Correct Answer: (A)
Explanation: re.sub replaces all whitespace sequences (\s+) with a single hyphen.

Q262. What is printed?

Iterators

Medium

Output

```
from itertools import count
c = count(start=5, step=3)
print(next(c), next(c), next(c))
```

[OK] (A) 5 8 11

(B) 5 6 7

(C) 3 6 9

(D) Error

Correct Answer: (A)
Explanation: count(5, 3) generates 5, 8, 11, 14, ... incrementing by 3 each time.

Q263. What is the output?

Data Types

Medium

Output

```
x = 5
print(x << 2)
```

[OK] (A) 20

(B) 10

(C) 25

(D) 2

Correct Answer: (A)
*Explanation: Left shift by 2 multiplies by 4. 5 << 2 = 5 * 4 = 20.*

Q264. What does this code print?

Strings

Easy

Output

```
print('Hello World'.count('o'))
```

[OK] (A) 2

(B) 1

(C) 3

(D) 0

Correct Answer: (A)
Explanation: str.count('o') counts non-overlapping occurrences. 'o' appears twice in 'Hello World'.

Q265. What is the output?

Lists

Easy

Output

```
lst = [1, 2, 3]
lst.reverse()
print(lst)
```

[OK] (A) [3, 2, 1]

(B) [1, 2, 3]

(C) None

(D) Error

Correct Answer: (A)
Explanation: list.reverse() reverses the list in-place and returns None. The list itself becomes [3, 2, 1].

Q266. What does this print?

Dicts

Easy

Output

```
d = {'a': 1, 'b': 2}
print(list(d.keys()))
```

[OK] (A) ['a', 'b']

(B) [1, 2]

(C) [('a',1), ('b',2)]

(D) Error

Correct Answer: (A)
Explanation: dict.keys() returns a view of keys. list() converts it to ['a', 'b'].

Q267. What is the output?

Functions

Easy

Output

```
def multiply(*args):
result = 1
for a in args:
result *= a
return result
print(multiply(2, 3, 4))
```

[OK] (A) 24

(B) 9

(C) 234

(D) Error

Correct Answer: (A)
*Explanation: *args collects all positional arguments. The function multiplies 2 * 3 * 4 = 24.*

Q268. What does this code print?

OOP

Easy

Output

```
class Counter:
count = 0
def __init__(self):
Counter.count += 1
a = Counter()
b = Counter()
print(Counter.count)
```

[OK] (A) 2

(B) 1

(C) 0

(D) Error

Correct Answer: (A)
Explanation: Each instantiation increments the class attribute. After two instances, Counter.count is 2.

Q269. What is the output?

Comprehensions

Easy

Output

```
print({x for x in [1, 2, 2, 3, 3, 3]})
```

[OK] (A) {1, 2, 3}

(B) [1, 2, 3]

(C) {1, 2, 2, 3, 3, 3}

(D) Error

Correct Answer: (A)
Explanation: A set comprehension removes duplicates, producing {1, 2, 3}.

Exception Handling

Q270. What is the output?

Hard

Output

```
def f():
try:
return 1
finally:
return 2
print(f())
```

[OK] (A) 2

(B) 1

(C) None

(D) Error

Correct Answer: (A)
Explanation: The finally block's return overrides the try block's return. The function returns 2.

Q271. What does this code print?

Decorators

Medium

Output

```
def trace(func):
def wrapper(*args):
print(f'calling {func.__name__}')
return func(*args)
return wrapper

@trace
def hello():
print('hello!')
hello()
```

[OK] (A) calling hello\nhello!

(B) hello!\ncalling hello

(C) hello!

(D) Error

Correct Answer: (A)

Explanation: The decorator prints the function name before calling it. First 'calling hello' then 'hello!'.

Q272. What is printed?

Sets

Medium

Output

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}
print(a.intersection(b))
```

[OK] (A) {3, 4}

(B) {1, 2, 3, 4, 5, 6}

(C) {1, 2}

(D) {5, 6}

Correct Answer: (A)
Explanation: intersection() returns elements common to both sets: {3, 4}.

Q273. What is the output?

Tuples

Easy

Output

```
t = (1,)
print(type(t))
```

[OK] (A)

(B)

(C)

(D) Error

Correct Answer: (A)
Explanation: The trailing comma makes (1,) a tuple, not just the integer 1 in parentheses.

Q274. What does this code print?

Scope

Easy

Output

```
x = 'outer'
def f():
print(x)
f()
```

[OK] (A) outer

(B) None

(C) Error

(D) inner

Correct Answer: (A)
Explanation: x is not defined locally in f(), so Python looks up to the global scope and finds x = 'outer'.

Q275. What is the output?

Inheritance

Easy

Output

```
class A:
pass
class B(A):
pass
print(issubclass(B, A))
```

[OK] (A) TRUE

(B) FALSE

(C) Error

(D) None

Correct Answer: (A)
Explanation: issubclass(B, A) returns True because B inherits from A.

Q276. What is printed?

Built-ins

Easy

Output

```
print(min([5, 3, 8, 1, 9]))
```

[OK] (A) 1

(B) 5

(C) 9

(D) Error

Correct Answer: (A)
Explanation: min() returns the smallest element from the iterable. The minimum of [5, 3, 8, 1, 9] is 1.

Q277. What does this code print?

Advanced Python

Easy

Output

```
x = [1, 2, 3]
print(x[::-1])
```

[OK] (A) [3, 2, 1]

(B) [1, 2, 3]

(C) [1, 3]

(D) Error

Correct Answer: (A)
Explanation: Slice [::-1] reverses any sequence. It creates a new reversed list.

Q278. What is the output?

Closures

Easy

Output

```
def outer():
msg = 'hello'
def inner():
return msg
return inner
f = outer()
print(f())
```

[OK] (A) hello

(B) None

(C) Error

(D) outer

Correct Answer: (A)
Explanation: inner() is a closure that captures msg from the enclosing scope. When called, it returns 'hello'.

Q279. What is the output?

Modules

Medium

Output

```
import math
print(math.ceil(4.1))
```

[OK] (A) 5

(B) 4

(C) 4.1

(D) Error

Correct Answer: (A)
Explanation: math.ceil() returns the smallest integer greater than or equal to the argument. ceil(4.1) = 5.

Q280. What is printed?

Generators

Easy

Output

```
g = (x for x in range(3))
print(type(g))
```

[OK] (A)

(B)

(C)

(D) Error

Correct Answer: (A)
Explanation: Parentheses around a comprehension create a generator expression, which is of type 'generator'.

Q281. What is the output?

Regular Expressions

Easy

Output

```
import re
result = re.split(r'[,;]', 'a,b;c,d')
print(result)
```

[OK] (A) ['a', 'b', 'c', 'd']

(B) ['a,b;c,d']

(C) ['a', 'b;c', 'd']

(D) Error

Correct Answer: (A)
Explanation: re.split with character class [,;] splits on either comma or semicolon, producing four parts.

Q282. What does this code print?

File I/O

Medium

Output

```
import io
buf = io.BytesIO(b'hello')
print(buf.read())
```

[OK] (A) b'hello'

(B) hello

(C) ['h','e','l','l','o']

(D) Error

Correct Answer: (A)
Explanation: BytesIO is an in-memory binary stream. read() returns the bytes object b'hello'.

Q283. You need to store a collection of unique user IDs that supports fast membership testing. Which data structure is most appropriate?

Data Types

Medium

Scenario

[OK] (A) set

(B) list

(C) tuple

(D) string

Correct Answer: (A)
Explanation: Sets provide O(1) average-case membership testing and automatically enforce uniqueness.

Q284. A user inputs ' Hello World '. You need to remove extra whitespace from both ends. Which method should you use?

Strings

Easy

Scenario

[OK] (A) strip()

(B) replace()

(C) split()

(D) center()

Correct Answer: (A)
Explanation: strip() removes leading and trailing whitespace, turning ' Hello World ' into 'Hello World'.

Q285. You need to process a list of 10 million records and extract only those matching a condition. What is the most memory-efficient approach?

Lists

Medium

Scenario

[OK] (A) Generator expression

(B) List comprehension

(C) map() with list()

(D) Nested for loops

Correct Answer: (A)
Explanation: A generator expression processes items lazily one at a time, avoiding loading all results into memory.

Q286. You need to count word frequencies in a document. Which approach is most Pythonic?

Dicts

Medium

Scenario

[OK] (A) collections.Counter

(B) Manual dict with if/else

(C) list of tuples

(D) sorted list

Correct Answer: (A)
Explanation: Counter is purpose-built for counting. Counter(words) creates a frequency dictionary in one line.

Q287. You have two lists of email addresses and need to find emails that appear in both lists. What operation should you use?

Sets

Easy

Scenario

[OK] (A) Set intersection

(B) List concatenation

(C) Dictionary merge

(D) String matching

Correct Answer: (A)
Explanation: Converting to sets and using intersection (&) efficiently finds common elements.

Q288. You want to return multiple values from a function (a name and an age). What is the most Pythonic approach?

Tuples

Easy

Scenario

[OK] (A) Return a tuple

(B) Use global variables

(C) Return a formatted string

(D) Write to a file

Correct Answer: (A)
Explanation: Returning a tuple (name, age) is idiomatic Python. The caller can unpack: name, age = get_person().

Q289. You are writing a function that should accept any number of keyword arguments to configure settings. What parameter should you use?

Functions

Medium

Scenario

[OK] (A) **kwargs

(B) *args

(C) A fixed list of parameters

(D) Global variables

Correct Answer: (A)
*Explanation: **kwargs collects arbitrary keyword arguments into a dictionary, perfect for flexible configuration.*

Q290. You need to ensure that a class attribute is read-only from outside the class. What should you use?

OOP

Medium

Scenario

[OK] (A) @property with only a getter

(B) A global variable

(C) A class method

(D) A static variable

Correct Answer: (A)
Explanation: @property without a setter makes the attribute read-only. Any attempt to set it raises AttributeError.

Q291. You are designing a plugin system where plugins share some common behavior but each has unique logic. What pattern is best?

Inheritance

Hard

Scenario

[OK] (A) Abstract Base Class with abstract methods

(B) Copy-paste common code

(C) Monkey patching at runtime

(D) Global functions

Correct Answer: (A)
Explanation: An ABC defines the interface and common behavior. Each plugin subclass implements the abstract methods.

Q292. You want to add logging to 20 different functions without modifying their code. What approach is best?

Decorators

Medium

Scenario

[OK] (A) Create a logging decorator

(B) Add print statements to each function

(C) Use a logging config file

(D) Subclass each function

Correct Answer: (A)
Explanation: A decorator cleanly adds logging behavior. Apply @log_decorator to each function with minimal code changes.

Q293. You need to process a 50GB log file line by line. What approach avoids loading the entire file into memory?

Generators

Hard

Scenario

[OK] (A) A generator that yields one line at a time

(B) read() the entire file

(C) readlines() into a list

(D) Load into a pandas DataFrame

Correct Answer: (A)
Explanation: A generator (or iterating the file object) yields one line at a time, using constant memory regardless of file size.

Q294. You need to create an object that generates prime numbers on demand. What should you implement?

Iterators

Medium

Scenario

[OK] (A) The iterator protocol (__iter__ and __next__)

(B) A list of all primes

(C) A recursive function

(D) A global counter

Correct Answer: (A)
Explanation: Implementing __iter__ and __next__ makes a lazy prime generator that produces primes one at a time.

Q295. You need to create a list of squares of even numbers from 0 to 20. What is the most Pythonic approach?

Comprehensions

Easy

Scenario

[OK] (A) [x2 for x in range(21) if x % 2 == 0]**

(B) for loop with if and append

(C) map with filter and list

(D) while loop with counter

Correct Answer: (A)
Explanation: List comprehensions with filtering are the most Pythonic way to create transformed, filtered lists.

Q296. Your function connects to a database, performs a query, and must close the connection even if an error occurs. What pattern should you use?

Exception Handling

Medium

Scenario

[OK] (A) try/finally or a context manager

(B) try/except only

(C) Global connection variable

(D) Ignore the error

Correct Answer: (A)
Explanation: try/finally ensures cleanup (closing the connection) regardless of exceptions. A context manager is even better.

Q297. You need to read a CSV file and process each row. What module should you use?

File I/O

Easy

Scenario

[OK] (A) csv

(B) json

(C) pickle

(D) xml

Correct Answer: (A)
Explanation: The csv module provides readers and writers specifically designed for CSV file handling.

Q298. You are building a large application and want to organize code into multiple files with shared utilities. What structure should you use?

Modules

Medium

Scenario

[OK] (A) A Python package with __init__.py and submodules

(B) A single large file

(C) Global variables across files

(D) Shell scripts for organization

Correct Answer: (A)
Explanation: Packages provide hierarchical module organization with __init__.py files defining the package structure.

Q299. You need to validate that a string is a valid email address format. What is the recommended approach?

Regular Expressions

Medium

Scenario

[OK] (A) Use re.match() with an email pattern

(B) Check for '@' character only

(C) Try to send an email

(D) Use string.find()

Correct Answer: (A)
Explanation: A regex pattern can validate email format (e.g., r'^[\w.+-]+@[\w-]+\.[\w.]+$') for basic validation.

Q300. You need to sort a list of dictionaries by a specific key. What is the most Pythonic approach?

Built-ins | Easy | Scenario

[OK] (A) sorted() with a key function

(B) Manual bubble sort

(C) Convert to tuples first

(D) Use collections.OrderedDict

Correct Answer: (A)
Explanation: sorted(dicts, key=lambda d: d['age']) sorts by the specified key efficiently and Pythonically.

Q301. You need a dictionary that returns 0 for missing keys instead of raising KeyError. What should you use?

Collections | Medium | Scenario

[OK] (A) defaultdict(int)

(B) Regular dict with try/except

(C) OrderedDict

(D) ChainMap

Correct Answer: (A)
Explanation: defaultdict(int) automatically creates missing keys with a default value of int() = 0.

Q302. You have a configuration value that multiple functions in a module need to access. What is the cleanest approach?

Scope

Medium

Scenario

(A) Module-level constant

[OK] (B) Pass as argument to each function

(C) Global variable modified by each function

(D) Environment variable

Correct Answer: (B)
Explanation: Passing configuration as a parameter is the cleanest approach. It makes dependencies explicit and aids testing.

Q303. You need to create a family of functions that each multiply by a different factor. What pattern should you use?

Closures

Hard

Scenario

(A) Global variables for each factor

(B) Separate function definitions for each

[OK] (C) A closure factory function

(D) Class with different methods

Correct Answer: (C)
*Explanation: A closure factory (def multiplier(n): return lambda x: x*n) generates customized functions sharing the pattern.*

Q304. You are designing a class that controls how its instances are created (e.g., singleton pattern). What Python feature should you use?

Advanced Python

Hard

Scenario

(A) Global variable check

(B) __init__ override

[OK] (C) __new__ method override

(D) Module-level function

Correct Answer: (C)
Explanation: __new__ controls instance creation. For singleton, it can return an existing instance instead of creating a new one.

Q305. You need to store a constant mathematical value like pi. What data type is appropriate?

Data Types

Easy

Scenario

(A) str

(B) int

[OK] (C) float

(D) bool

Correct Answer: (C)
Explanation: Pi (3.14159...) is a decimal number, so float is the appropriate type. For higher precision, use decimal.Decimal.

Q306. You need to build a SQL query string with user inputs safely. What approach should you use?

Strings

Medium

Scenario

(A) String concatenation with +

(B) f-strings directly

[OK] (C) Parameterized queries with placeholders

(D) str.format()

Correct Answer: (C)
Explanation: Parameterized queries prevent SQL injection. Never use string formatting with user input in SQL queries.

Q307. You need a data structure supporting fast O(1) appends and pops from both ends. What should you use?

Lists

Hard

Scenario

(A) list

(B) tuple

[OK] (C) collections.deque

(D) set

Correct Answer: (C)
Explanation: deque provides O(1) operations on both ends. Lists are O(n) for operations at the front.

Q308. You need to merge multiple configuration dictionaries where later ones override earlier values. What Python 3.9+ approach is cleanest?

Dicts

Hard

Scenario

(A) Manual loop

(B) dict.update() chain

[OK] (C) Using the | operator to merge dicts

(D) json.loads()

Correct Answer: (C)
Explanation: In Python 3.9+, d1 | d2 | d3 cleanly merges dictionaries, with later ones taking precedence.

Q309. You have a computationally expensive pure function called repeatedly with the same arguments. How do you optimize it?

Functions

Hard

Scenario

(A) Rewrite in C

(B) Use global cache dict

[OK] (C) Use @functools.lru_cache

(D) Use multithreading

Correct Answer: (C)
Explanation: lru_cache automatically caches function results based on arguments, avoiding redundant computations.

Q310. You want to create a class that behaves like a dictionary but adds validation on key assignment. What approach is best?

OOP

Hard

Scenario

(A) Monkey patch dict

(B) Wrap dict in a tuple

[OK] (C) Inherit from dict and override __setitem__

(D) Use a list of tuples

Correct Answer: (C)
Explanation: Subclassing dict and overriding __setitem__ lets you add validation while preserving all dict functionality.

Q311. You need a decorator that retries a function up to 3 times on failure with exponential backoff. What structure do you need?

Decorators

Hard

Scenario

(A) A simple decorator

(B) A class-based decorator

[OK] (C) A decorator factory that accepts retry count

(D) A context manager

Correct Answer: (C)
Explanation: A decorator factory allows parameterization: @retry(max_attempts=3, backoff=2) passes configuration to the decorator.

Q312. You want to create an infinite sequence of Fibonacci numbers that consumers can take from as needed. What should you use?

Generators

Medium

Scenario

(A) A list of pre-computed values

(B) A recursive function

[OK] (C) A generator with yield

(D) A class with __call__

Correct Answer: (C)
Explanation: A generator with yield lazily produces values on demand, perfect for potentially infinite sequences.

Q313. You are building a library and need to define domain-specific errors. What is the best practice?

Exception Handling

Hard

Scenario

(A) Use ValueError for everything

(B) Use string error messages

[OK] (C) Create a custom exception hierarchy inheriting from Exception

(D) Use sys.exit() codes

Correct Answer: (C)
Explanation: A custom exception hierarchy (e.g., LibraryError -> DatabaseError, ValidationError) gives users precise error handling.

File I/O

Medium

Scenario

Q314. You need to parse a large JSON file that might not fit in memory. What approach should you use?

(A) json.loads() on the entire file

(B) Read line by line

[OK] (C) Use ijson for iterative JSON parsing

(D) Convert to CSV first

Correct Answer: (C)
Explanation: ijson provides iterative JSON parsing, processing the file incrementally without loading it entirely into memory.

Modules

Hard

Scenario

Q315. You need to dynamically load a module based on a string name at runtime. What should you use?

(A) exec()

(B) eval()

[OK] (C) importlib.import_module()

(D) __import__()

Correct Answer: (C)
Explanation: importlib.import_module(name) is the recommended way to dynamically import modules by name.

Q316. You need to extract all URLs from a large text document efficiently. What approach is best?

Regular Expressions

Hard

Scenario

(A) String split and filter

(B) Manual character scanning

[OK] (C) re.findall() with a URL pattern

(D) String replace

Correct Answer: (C)
Explanation: re.findall() with a URL regex efficiently extracts all matching URLs from the text.

Q317. You need to check if all elements in a list satisfy a condition. What built-in function should you use?

Built-ins

Medium

Scenario

(A) any()

(B) filter()

[OK] (C) all()

(D) map()

Correct Answer: (C)
Explanation: all(condition(x) for x in lst) returns True only if every element satisfies the condition.

Q318. You need a dictionary that searches multiple scopes (local, global, defaults) in order. What should you use?

Collections

Hard

Scenario

(A) Nested dicts

(B) defaultdict

[OK] (C) collections.ChainMap

(D) dict.update() merge

Correct Answer: (C)
Explanation: ChainMap groups multiple dicts and searches them in order, perfect for scoped lookups.

Q319. You want to implement a simple caching mechanism for a pure function without using decorators. What approach uses closures?

Closures

Medium

Scenario

(A) Global dictionary

(B) Class with __call__

[OK] (C) A function that returns a function with an enclosed cache dict

(D) Module-level cache

Correct Answer: (C)
Explanation: A closure can enclose a cache dictionary: def cached(f): cache = {}; def wrapper(x): ... return wrapper.

Q320. A junior developer's function accidentally modifies a global list. How do you prevent this?

Scope

Hard

Scenario

(A) Use a set instead

(B) Use a tuple instead

[OK] (C) Pass a copy of the list to the function

(D) Use a class method

Correct Answer: (C)
Explanation: Passing a copy (func(lst.copy())) ensures the function cannot modify the original list.

Q321. You need to make your custom class work with the 'with' statement. What should you implement?

Advanced Python

Medium

Scenario

(A) __init__ and __del__

(B) __enter__ only

[OK] (C) __enter__ and __exit__

(D) __start__ and __stop__

Correct Answer: (C)
Explanation: The context manager protocol requires __enter__ (setup) and __exit__ (cleanup) methods.

Q322. You have a Shape base class and want to ensure all subclasses implement an area() method. What should you do?

Inheritance

Medium

Scenario

(A) Document the requirement

(B) Raise NotImplementedError

[OK] (C) Use ABC with @abstractmethod

(D) Check at runtime

Correct Answer: (C)
Explanation: Using ABC with @abstractmethod prevents instantiation of subclasses that don't implement area().

Q323. You need to iterate over a large database result set without loading all rows. What pattern should you use?

Iterators

Hard

Scenario

(A) fetchall() into a list

(B) Read all into pandas

[OK] (C) Server-side cursor with iterator protocol

(D) JSON dump

Correct Answer: (C)
Explanation: A server-side cursor with iterator protocol fetches rows in batches, keeping memory usage constant.

Q324. You need to perform exact decimal arithmetic for a financial application. What should you use?

Data Types

Hard

Scenario

(A) float

(B) int

(C) numpy.float64

[OK] (D) decimal.Decimal

Correct Answer: (D)
Explanation: decimal.Decimal provides exact decimal representation, avoiding floating-point errors critical in finance.

Q325. You need to efficiently build a string from thousands of small pieces. What is the fastest approach?

Strings

Hard

Scenario

(A) String concatenation with +=

(B) f-strings in a loop

(C) String multiplication

[OK] (D) ".join() on a list of parts

Correct Answer: (D)
Explanation: ".join(parts) builds the final string in one operation, avoiding O(n^2) cost of repeated concatenation.

Q326. You need to write a function that accepts either a filename or a file object. What feature helps?

Functions

Easy

Scenario

(A) Overloading

(B) Decorators

(C) Metaclasses

[OK] (D) Duck typing - check for read() method

Correct Answer: (D)
Explanation: Duck typing: check if the argument has a read() method (file-like object) rather than checking its type.

Q327. You want to track how many instances of a class have been created. Where should you store the count?

OOP

Easy

Scenario

(A) In __init__

(B) In a local variable

(C) In a global variable

[OK] (D) As a class attribute

Correct Answer: (D)
Explanation: A class attribute is shared across all instances and persists. Increment it in __init__ to track instance count.

Q328. You need to process items from a list but want to transform each item lazily. What should you use?

Generators

Easy

Scenario

(A) for loop with list

(B) list comprehension

(C) sorted()

[OK] (D) Generator expression

Correct Answer: (D)
*Explanation: A generator expression (x*2 for x in items) transforms items lazily, one at a time, saving memory.*

Q329. You want to ensure a database connection is always closed, even if an exception occurs. What pattern is best?

Exception Handling

Easy

Scenario

(A) try/except only

(B) Manual close() call

(C) atexit handler

[OK] (D) with statement (context manager)

Correct Answer: (D)
Explanation: The with statement automatically calls __exit__ which closes the connection, even if an exception occurs.

Q330. You need to atomically write to a file to prevent corruption if the process crashes mid-write. What approach is best?

File I/O

Hard

Scenario

(A) Direct write with flush()

(B) Use a lock file

(C) Append mode only

[OK] (D) Write to a temp file and rename

Correct Answer: (D)
Explanation: Writing to a temporary file and atomically renaming it ensures the original file is never in a partial state.

Q331. You want to make your Python script both importable as a module and runnable as a standalone script. What guard should you use?

Modules

Easy

Scenario

(A) try/except ImportError

(B) sys.argv check

(C) os.environ check

[OK] (D) if __name__ == '__main__':

Correct Answer: (D)
Explanation: The __name__ guard runs code only when the script is executed directly, not when imported.

Q332. You need a FIFO queue with O(1) operations on both ends. What should you use?

Collections

Easy

Scenario

(A) list

(B) tuple

(C) set

[OK] (D) collections.deque

Correct Answer: (D)
Explanation: deque (double-ended queue) provides O(1) append and popleft, ideal for FIFO queues.

Q333. You need to flatten a list of lists. What is the most Pythonic one-liner using built-ins?

Built-ins

Hard

Scenario

(A) reduce()

(B) map()

(C) filter()

[OK] (D) sum(lists, []) or itertools.chain.from_iterable()

Correct Answer: (D)
Explanation: itertools.chain.from_iterable(lists) efficiently flattens nested iterables. sum(lists, []) works but is O(n^2).

Q334. You want to add type hints and validate data at runtime for API input. What tool is best?

Advanced Python

Hard

Scenario

(A) Manual isinstance() checks

(B) assert statements

(C) typing module only

[OK] (D) pydantic or dataclass with validation

Correct Answer: (D)
Explanation: Pydantic provides runtime data validation using type hints, auto-parsing, and clear error messages.

Q335. You need to split a string on multiple delimiters (comma, semicolon, pipe). What should you use?

Regular Expressions

Easy

Scenario

(A) str.split()

(B) str.partition()

(C) str.replace()

[OK] (D) re.split(r'[,;|]', text)

Correct Answer: (D)
Explanation: re.split() with a character class [,;|] splits on any of the specified delimiters.

Q336. You need to iterate over both keys and values of a dictionary simultaneously. What method should you use?

Dicts

Easy

Scenario

(A) keys()

(B) values()

(C) get()

[OK] (D) items()

Correct Answer: (D)
Explanation: dict.items() returns (key, value) pairs: for k, v in d.items(): ...

Q337. You want to define a constant used throughout your module. Where should you define it?

Scope

Easy

Scenario

(A) Inside a function

(B) Inside a class

(C) In a separate config class

[OK] (D) At module level with UPPER_CASE name

Correct Answer: (D)
Explanation: Module-level constants in UPPER_CASE (e.g., MAX_SIZE = 100) is the Python convention for constants.

Q338. You have two large datasets and need to find items that appear in only one of them. What set operation should you use?

Sets

Medium

Scenario

(A) Union

(B) Intersection

(C) Difference

[OK] (D) Symmetric difference

Correct Answer: (D)
Explanation: Symmetric difference (a ^ b) returns elements in either set but not both, finding unique-to-each items.

Q339. You need to use a pair of coordinates as a dictionary key. Which data type should you use?

Tuples

Medium

Scenario

(A) list

(B) set

(C) dict

[OK] (D) tuple

Correct Answer: (D)
Explanation: Tuples are immutable and hashable, making them suitable as dictionary keys. Lists are not hashable.

Q340. You have a logging mixin, a caching mixin, and a base class. How should you combine them?

Inheritance

Hard

Scenario

(A) Nested inheritance chain

(B) Composition only

(C) Copy methods manually

[OK] (D) Multiple inheritance: class MyClass(LogMixin, CacheMixin, Base)

Correct Answer: (D)
Explanation: Multiple inheritance with mixins cleanly combines orthogonal behaviors without deep inheritance hierarchies.

Q341. You want to measure the execution time of a function. What is the cleanest approach?

Decorators

Easy

Scenario

(A) Manual time.time() before and after

(B) Print statements

(C) Profiler module

[OK] (D) A timing decorator

Correct Answer: (D)
Explanation: A decorator wraps the function with timing logic, keeping the function code clean and reusable.

Q342. You need to process pairs of adjacent elements from a list. What itertools function helps?

Iterators

Easy

Scenario

(A) chain()

(B) count()

(C) cycle()

[OK] (D) pairwise() (Python 3.10+)

Correct Answer: (D)
Explanation: itertools.pairwise() yields consecutive overlapping pairs from an iterable: (s[0],s[1]), (s[1],s[2]), etc.

Q343. You need to create a dictionary mapping filenames to their extensions. What is the most Pythonic approach?

Comprehensions

Medium

Scenario

(A) Manual for loop

(B) map() with lambda

(C) filter() with lambda

[OK] (D) Dict comprehension: {f: f.split('.')[-1] for f in files}

Correct Answer: (D)
Explanation: A dict comprehension is the most concise and Pythonic way to create the mapping.

Q344. You want to create a function that remembers how many times it has been called. What approach uses closures?

Closures

Easy

Scenario

(A) Global counter variable

(B) Class with __call__

(C) Static method

[OK] (D) Nested function with nonlocal counter

Correct Answer: (D)
Explanation: A closure with nonlocal counter keeps the count in the enclosing scope, avoiding global state.

Q345. You need to process a very large CSV file (100GB) row by row. What is the most memory-efficient approach?

File I/O

Hard

Scenario

(A) pandas.read_csv()

(B) csv.reader() with readlines()

(C) json.load()

[OK] (D) csv.reader() iterating the file object directly

Correct Answer: (D)
Explanation: Iterating the file object with csv.reader reads one row at a time from disk, using constant memory.

Q346. You need to serialize Python objects to binary format for inter-process communication. What should you use?

Advanced Python

Medium

Scenario

(A) json

(B) csv

(C) xml

[OK] (D) pickle (with security considerations)

Correct Answer: (D)
Explanation: pickle serializes arbitrary Python objects to binary. Use with trusted data only due to security risks.

Q347. You need to remove duplicates from a list while preserving order. What approach works in Python 3.7+?

Built-ins

Easy

Scenario

(A) sorted(set(lst))

(B) list(set(lst))

(C) Using a loop with 'in' check

[OK] (D) list(dict.fromkeys(lst))

Correct Answer: (D)
Explanation: dict.fromkeys(lst) preserves insertion order (Python 3.7+) and removes duplicates. Converting back to list gives the result.

Q348. You need to represent a fraction 1/3 exactly without floating-point error. What module should you use?

Data Types

Hard

Scenario

(A) math

(B) decimal

(C) numpy

[OK] (D) fractions

Correct Answer: (D)
Explanation: The fractions module provides a Fraction class that represents rational numbers exactly. Fraction(1, 3) is precise.

Q349. You need to check if a user-submitted password contains at least one uppercase letter. What method helps?

Strings

Easy

Scenario

(A) isupper()

(B) islower()

[OK] (C) any(c.isupper() for c in password)

(D) upper()

Correct Answer: (C)
Explanation: Using any() with a generator checking each character's case is the Pythonic way to verify mixed-case requirements.

Q350. You need to check if a list is empty before processing it. What is the most Pythonic way?

Lists

Easy

Scenario

(A) if len(lst) == 0:

(B) if lst == []:

[OK] (C) if not lst:

(D) if lst is None:

Correct Answer: (C)
Explanation: 'if not lst:' is the most Pythonic idiom. Empty lists are falsy, so 'not []' evaluates to True.

Q351. You need to iterate over a dictionary and print each key-value pair. What is the most Pythonic approach?

Dicts

Easy

Scenario

(A) for k in d: print(k, d[k])

(B) for i in range(len(d)):

[OK] (C) for k, v in d.items():

(D) while loop with pop()

Correct Answer: (C)
Explanation: for k, v in d.items(): unpacks each key-value pair directly, which is the most readable and Pythonic approach.

Q352. You want a function to accept configuration options as keyword arguments and pass them to another function. What parameter should you use?

Functions

Medium

Scenario

(A) *args

(B) Named parameters

[OK] (C) **kwargs

(D) A config object

Correct Answer: (C)
*Explanation: **kwargs collects keyword arguments into a dict. You can then pass them with **kwargs to another function.*

Q353. You want to prevent a class from being modified after creation. Which approach helps?

OOP

Easy

Scenario

(A) Use global variables

(B) Use a list

[OK] (C) Use __slots__ and make attributes read-only

(D) Use a tuple

Correct Answer: (C)
Explanation: __slots__ restricts attributes and combined with @property (getters only) creates a mostly immutable class.

Q354. You have common logging code used by multiple unrelated classes. What pattern is best?

Inheritance

Easy

Scenario

(A) Duplicate the code

(B) Global functions

[OK] (C) A mixin class

(D) Decorator on the module

Correct Answer: (C)
Explanation: A LoggingMixin class provides logging methods. Classes can inherit from it alongside their main base class.

Q355. You want to enforce that a function argument is always a positive integer. What approach is cleanest?

Decorators

Medium

Scenario

(A) Add checks at every call site

(B) Use global validation

[OK] (C) A validation decorator

(D) String assertions

Correct Answer: (C)
Explanation: A decorator can validate arguments before calling the function, keeping the function code clean and the validation reusable.

Q356. You need to implement pagination over a large dataset, yielding one page at a time. What is the best approach?

Generators

Medium

Scenario

(A) Load all pages at once

(B) Use recursion

[OK] (C) A generator that yields page-sized chunks

(D) Create separate functions per page

Correct Answer: (C)
Explanation: A generator yielding slices of the dataset provides lazy, memory-efficient pagination.

Q357. You need to combine multiple sorted iterators into a single sorted iterator. What should you use?

Iterators

Medium

Scenario

(A) sorted() on concatenated lists

(B) list.sort()

[OK] (C) heapq.merge()

(D) itertools.chain()

Correct Answer: (C)
*Explanation: heapq.merge(*iterables) merges multiple sorted inputs into a single sorted output, lazily and efficiently.*

Q358. You need to transpose a matrix (list of lists). What is the most Pythonic one-liner?

Comprehensions

Hard

Scenario

(A) Manual nested loops

(B) numpy.transpose()

[OK] (C) list(zip(*matrix))

(D) matrix.T

Correct Answer: (C)
*Explanation: zip(*matrix) unpacks rows and zips columns together, effectively transposing the matrix.*

Q359. Your code might divide by zero. You want to handle it gracefully and return 0 instead. What pattern should you use?

Exception Handling

Easy

Scenario

(A) Check with assert

(B) Use a global flag

[OK] (C) try/except ZeroDivisionError

(D) Ignore the error

Correct Answer: (C)
Explanation: try/except ZeroDivisionError catches the specific error and allows returning a default value.

Q360. You need to check if a file exists before opening it. What module provides the most Pythonic approach?

File I/O

Easy

Scenario

(A) os.system

(B) shutil

[OK] (C) pathlib

(D) sys

Correct Answer: (C)
Explanation: Path('file.txt').exists() from pathlib provides an object-oriented way to check file existence.

Q361. You need to package your Python project for distribution on PyPI. What tool should you use?

Modules

Medium

Scenario

(A) pip only

(B) conda only

[OK] (C) setuptools with pyproject.toml

(D) manual zip files

Correct Answer: (C)
Explanation: setuptools with pyproject.toml is the standard way to package Python projects for distribution.

Q362. You need to validate that a string is a valid Python variable name. What is the simplest approach?

Regular Expressions

Medium

Scenario

(A) Manual character checks

(B) Try to assign it

[OK] (C) str.isidentifier()

(D) regex pattern

Correct Answer: (C)
Explanation: str.isidentifier() checks if the string is a valid Python identifier. Combined with keyword.iskeyword() for full validation.

Q363. You need to transform a list of strings to integers, filtering out invalid ones. What is the cleanest approach?

Built-ins

Medium

Scenario

(A) Manual try/except in a loop

(B) regex validation

[OK] (C) A list comprehension with a helper function and try/except

(D) str.isdigit() check

Correct Answer: (C)
Explanation: A comprehension with a helper that catches ValueError provides clean filtering: [safe_int(s) for s in items if safe_int(s) is not None].

Q364. You need an ordered dictionary that also provides fast access to the most and least recently added items. What should you use?

Collections

Medium

Scenario

(A) dict

(B) defaultdict

[OK] (C) OrderedDict

(D) deque

Correct Answer: (C)
Explanation: OrderedDict provides move_to_end() and popitem(last=True/False) for manipulating order, beyond what plain dict offers.

Q365. A team member accidentally shadows a built-in name (list = [1,2,3]). How do you still access the built-in?

Scope

Medium

Scenario

(A) Use List instead

(B) Restart Python

[OK] (C) import builtins; builtins.list

(D) Use [] syntax only

Correct Answer: (C)
Explanation: The builtins module holds all built-in names. builtins.list still references the original list type.

Q366. You want to create a configurable logger that remembers its log level. What pattern uses closures?

Closures

Medium

Scenario

(A) Global log level variable

(B) Class with __init__

[OK] (C) A factory function returning a closure over the level

(D) Module-level config

Correct Answer: (C)
Explanation: A factory function make_logger(level) returning a closure that captures level provides configurable, stateful loggers.

Q367. You need to profile a Python script to find performance bottlenecks. What built-in module should you use?

Advanced Python

Medium

Scenario

(A) timeit

(B) sys

[OK] (C) cProfile

(D) traceback

Correct Answer: (C)
Explanation: cProfile provides deterministic profiling of Python programs, showing time spent in each function call.

Q368. You need a lightweight class to represent database records with named fields. What is the most memory-efficient approach?

Tuples

Hard

Scenario

(A) Regular class

(B) Dictionary per record

[OK] (C) namedtuple or __slots__ class

(D) List per record

Correct Answer: (C)
Explanation: namedtuple creates tuple-like objects with named access. They use less memory than dicts or regular classes.

Q369. You need to efficiently check if two large lists have any elements in common. What approach is fastest?

Sets

Hard

Scenario

(A) Nested for loops

(B) sorted() and binary search

[OK] (C) not set(list1).isdisjoint(list2)

(D) Using == operator

Correct Answer: (C)
*Explanation: Converting to set and using isdisjoint() is O(min(n,m)) average case, much faster than O(n*m) nested loops.*

Q370. You need to store a very large integer (1000+ digits). What should you use?

Data Types

Medium

Scenario

(A) float

(B) numpy.int64

[OK] (C) Python's built-in int (arbitrary precision)

(D) ctypes.c_long

Correct Answer: (C)
Explanation: Python's int type supports arbitrary precision. It can handle integers of any size, limited only by available memory.

Q371. You need to find the top-k largest elements from a list of millions of items efficiently. What should you use?

Lists

Medium

Scenario

(A) sorted()[-k:]

(B) list.sort() then slice

[OK] (C) heapq.nlargest(k, lst)

(D) Manual iteration

Correct Answer: (C)
Explanation: heapq.nlargest(k, lst) is more efficient than sorting for small k and large lists, using a heap of size k.

Q372. You need to parse a complex text format with nested delimiters. What approach is most robust?

Strings

Hard

Scenario

(A) str.split()

(B) String slicing

[OK] (C) A proper parser (e.g., pyparsing or PLY)

(D) Regular expressions

Correct Answer: (C)
Explanation: For complex nested formats, a proper parser handles recursive structures that regex cannot. pyparsing is a good choice.

Q373. You need a cache with a maximum size that evicts the least recently used entries. What should you use?

Dicts

Medium

Scenario

(A) Regular dict

(B) defaultdict

[OK] (C) functools.lru_cache or OrderedDict-based LRU

(D) set

Correct Answer: (C)
Explanation: functools.lru_cache provides built-in LRU caching. For custom caches, OrderedDict's move_to_end enables LRU logic.

Q374. You need to document what a function does, its parameters, and return value. What Python feature should you use?

Functions

Easy

Scenario

(A) Comments above the function

(B) print statements

[OK] (C) A docstring (triple-quoted string as first statement)

(D) External documentation file

Correct Answer: (C)
Explanation: Docstrings are the standard way to document functions. They are accessible via help() and __doc__.

Q375. Complete the code to check if a variable x is of type int.

Data Types

Easy

Code Completion

```
x = 42
result = ___(x, int)
print(result) # True
```

(A) type()

[OK] (B) isinstance

(C) istype()

(D) check()

Correct Answer: (B)
Explanation: isinstance(x, int) checks if x is an instance of int or its subclasses. It is preferred over type(x) == int.

Q376. Complete the code to convert a float to an integer by truncating toward zero.

Data Types

Medium

Code Completion

```
import math
x = -3.7
result = math.___(x)
print(result) # -3
```

(A) floor

(B) ceil

[OK] (C) trunc

(D) round

Correct Answer: (C)
Explanation: math.trunc() removes the decimal part, truncating toward zero. floor(-3.7) would give -4.

Q377. Complete the code to join a list of words with a space.

Strings

Easy

Code Completion

```
words = ['Hello', 'World']
result = ___.___(words)
print(result) # Hello World
```

(A) words.join(' ')

(B) str.concat(words)

[OK] (C) ' '.join

(D) join(' ', words)

Correct Answer: (C)
Explanation: The join method is called on the separator string. ' '.join(['Hello', 'World']) produces 'Hello World'.

Q378. Complete the code to format a number with 2 decimal places.

Strings

Medium

Code Completion

```
price = 19.5
print(f'{price:___}')
```

(A) 2d

(B) %2f

[OK] (C) .2f

(D) 0.2

Correct Answer: (C)
Explanation: The format spec .2f means fixed-point notation with 2 decimal places. f'{19.5:.2f}' gives '19.50'.

Q379. Complete the code to remove all non-alphanumeric characters from a string.

Strings

Hard

Code Completion

```
import re
text = 'Hello, World! 123'
result = re.sub(r'___', '', text)
print(result) # HelloWorld123
```

(A) \d+

(B) \s+

[OK] (C) [^a-zA-Z0-9]

(D) [a-z]

Correct Answer: (C)
Explanation: [^a-zA-Z0-9] matches any character that is NOT alphanumeric. re.sub replaces all matches with empty string.

Lists

Easy

Code Completion

Q380. Complete the code to insert the value 99 at index 2.

```
lst = [1, 2, 3, 4]
lst.___(2, 99)
print(lst) # [1, 2, 99, 3, 4]
```

(A) append

(B) add

[OK] (C) insert

(D) push

Correct Answer: (C)
Explanation: list.insert(index, value) inserts the value before the given index position.

Lists

Medium

Code Completion

Q381. Complete the code to sort a list of tuples by the second element.

```
data = [(1, 'b'), (3, 'a'), (2, 'c')]
data.sort(key=lambda x: ___)
print(data) # [(3, 'a'), (1, 'b'), (2, 'c')]
```

(A) x

(B) x[0]

[OK] (C) x[1]

(D) x[-1]

Correct Answer: (C)
Explanation: x[1] extracts the second element of each tuple for comparison. Sorting by 'a','b','c' gives the expected order.

Q382. Complete the code to flatten a list of lists using itertools.

Lists

Hard

Code Completion

```
import itertools
nested = [[1, 2], [3, 4], [5, 6]]
flat = list(itertools.chain.___(*nested))
print(flat) # [1, 2, 3, 4, 5, 6]
```

(A) link

(B) merge

(C) connect

[OK] (D) from_iterable

Correct Answer: (D)
*Explanation: This uses chain(*nested) which unpacks the sublists. chain.from_iterable(nested) is the alternative without unpacking.*

Q383. Complete the code to get a value with a default of 'N/A' if the key is missing.

Dicts

Easy

Code Completion

```
d = {'name': 'Alice'}
age = d.___(___)
print(age) # N/A
```

(A) find('age', 'N/A')

(B) fetch('age', 'N/A')

(C) lookup('age', 'N/A')

[OK] (D) get('age', 'N/A')

Correct Answer: (D)
Explanation: dict.get(key, default) returns the default value when the key is not found, avoiding KeyError.

Dicts

Medium

Code Completion

Q384. Complete the code to merge two dictionaries in Python 3.9+.

```
d1 = {'a': 1}
d2 = {'b': 2}
merged = d1 ___ d2
print(merged) # {'a': 1, 'b': 2}
```

(A) +

(B) &

(C) ^

[OK] (D) |

Correct Answer: (D)
Explanation: The | operator merges dictionaries in Python 3.9+, returning a new dict with entries from both.

Sets

Easy

Code Completion

Q385. Complete the code to remove element 3 from the set without raising an error if it is missing.

```
s = {1, 2, 3}
s.___(3)
print(s)
```

(A) remove

(B) pop

(C) delete

[OK] (D) discard

Correct Answer: (D)
Explanation: discard() removes an element if present, silently doing nothing if absent. remove() raises KeyError.

Sets

Medium

Code Completion

Q386. Complete the code to check if set_a is a subset of set_b.

```
set_a = {1, 2}
set_b = {1, 2, 3, 4}
result = set_a.___(set_b)
print(result) # True
```

(A) contains

(B) includes

[OK] (C) issubset

(D) isin

Correct Answer: (C)
Explanation: set.issubset(other) returns True if all elements of the set are in the other set.

Tuples

Easy

Code Completion

Q387. Complete the code to find the index of value 30 in the tuple.

```
t = (10, 20, 30, 40)
i = t.___(30)
print(i) # 2
```

(A) find

(B) search

[OK] (C) index

(D) locate

Correct Answer: (C)
Explanation: tuple.index(value) returns the index of the first occurrence of the value.

Functions

Easy

Code Completion

Q388. Complete the function definition with a default parameter.

```
def greet(name___'World'):
return f'Hello, {name}!'
print(greet()) # Hello, World!
```

(A) :

(B) ==

[OK] (C) =

(D) ->

Correct Answer: (C)
Explanation: Default parameters use = sign: def greet(name='World'). When no argument is passed, 'World' is used.

Functions

Medium

Code Completion

Q389. Complete the code to create a partial function that always adds 10.

```
from functools import ___
def add(a, b):
return a + b
add10 = ___(add, 10)
print(add10(5)) # 15
```

(A) curry

(B) bind

[OK] (C) partial

(D) wrap

Correct Answer: (C)
*Explanation: functools.partial(func, *args) creates a new function with some arguments pre-filled. partial(add, 10) fixes a=10.*

Functions

Hard

Code Completion

Q390. Complete the code to apply a decorator that caches results.

```
from functools import ___

@___
def fibonacci(n):
if n < 2:
return n
return fibonacci(n-1) + fibonacci(n-2)
print(fibonacci(50))
```

(A) cache

(B) memoize

[OK] (C) lru_cache

(D) functools.cache

Correct Answer: (C)

Explanation: lru_cache (or cache in 3.9+) memoizes function results. Without it, fibonacci(50) would be impossibly slow.

OOP

Easy

Code Completion

Q391. Complete the constructor to initialize the name attribute.

```
class Person:
def ___(self, name):
self.name = name
p = Person('Alice')
print(p.name)
```

(A) __new__

(B) __create__

[OK] (C) __init__

(D) __start__

Correct Answer: (C)
Explanation: __init__ is the constructor method called when creating a new instance. It initializes instance attributes.

OOP

Medium

Code Completion

Q392. Complete the code to make the method a class method.

```
class MyClass:
count = 0
___
def get_count(cls):
return cls.count
```

(A) @staticmethod

(B) @abstract

[OK] (C) @classmethod

(D) @property

Correct Answer: (C)
Explanation: @classmethod binds the method to the class. It receives cls as the first argument instead of self.

OOP

Hard

Code Completion

Q393. Complete the code to implement the string representation.

```
class Point:
def __init__(self, x, y):
self.x = x
self.y = y
def ___(self):
return f'Point({self.x}, {self.y})'
p = Point(1, 2)
print(p) # Point(1, 2)
```

(A) __repr__

[OK] (B) __str__

(C) __format__

(D) __display__

Correct Answer: (B)
Explanation: __str__ defines the human-readable string representation used by print() and str().

Inheritance

Easy

Code Completion

Q394. Complete the code to call the parent class constructor.

```
class Animal:
def __init__(self, name):
self.name = name
class Dog(Animal):
def __init__(self, name, breed):
___.__init__(name)
self.breed = breed
```

(A) parent()

(B) base()

[OK] (C) super()

(D) Animal

Correct Answer: (C)
Explanation: super() returns a proxy object that delegates method calls to the parent class following the MRO.

Inheritance

Medium

Code Completion

Q395. Complete the code to define an abstract method.

```
from abc import ABC, ___
class Shape(ABC):
@___
def area(self):
pass
```

(A) virtual

(B) interface

[OK] (C) abstractmethod

(D) override

Correct Answer: (C)
Explanation: @abstractmethod from abc module declares a method that subclasses must implement.

Q396. Complete the decorator to preserve the wrapped function's metadata.

Decorators

Medium

Code Completion

```
from functools import ___
def my_decorator(func):
@___(func)
def wrapper(*args, **kwargs):
return func(*args, **kwargs)
return wrapper
```

(A) copy

(B) preserve

[OK] (C) wraps

(D) metadata

Correct Answer: (C)

Explanation: functools.wraps(func) copies __name__, __doc__, and other attributes from the original function to the wrapper.

Q397. Complete the generator function to yield even numbers.

Generators

Easy

Code Completion

```
def evens(n):
for i in range(n):
if i % 2 == 0:
___ i
print(list(evens(10)))
```

(A) return

(B) send

[OK] (C) yield

(D) output

Correct Answer: (C)
Explanation: yield produces a value and suspends the function. The generator yields each even number one at a time.

Q398. Complete the code to delegate to a sub-generator.

Generators

Hard

Code Completion

```
def chain_gen(*iterables):
for it in iterables:
___ it
print(list(chain_gen([1,2], [3,4])))
```

(A) return

(B) send

[OK] (C) yield from

(D) yield

Correct Answer: (C)
Explanation: 'yield from' delegates to the sub-iterator, yielding each of its values. Without 'from', it would yield the iterator itself.

Q399. Complete the iterator to stop after n items.

Iterators

Medium

Code Completion

```
class FirstN:
def __init__(self, data, n):
self.data = iter(data)
self.n = n
self.count = 0
def __iter__(self):
return self
def __next__(self):
if self.count >= self.n:
raise ___
self.count += 1
return next(self.data)
```

(A) IndexError

(B) ValueError

[OK] (C) StopIteration

(D) RuntimeError

Correct Answer: (C)

Explanation: StopIteration signals the end of iteration. Raising it tells the for loop (or list()) to stop consuming values.

Q400. Complete the set comprehension to get unique first letters.

Comprehensions

Easy

Code Completion

```
words = ['apple', 'banana', 'avocado', 'blueberry']
first_letters = {___}
print(first_letters) # {'a', 'b'}
```

(A) w for w in words

(B) w[1] for w in words

[OK] (C) w[0] for w in words

(D) len(w) for w in words

Correct Answer: (C)
Explanation: w[0] extracts the first character of each word. The set comprehension ensures uniqueness.

Q401. Complete the code to create a dictionary from two lists using a comprehension.

Comprehensions

Hard

Code Completion

```
keys = ['a', 'b', 'c']
vals = [1, 2, 3]
d = {k: v for k, v in ___(keys, vals)}
print(d)
```

(A) map

(B) enumerate

[OK] (C) zip

(D) chain

Correct Answer: (C)
Explanation: zip(keys, vals) pairs corresponding elements, which are then unpacked in the dict comprehension.

Exception Handling

Easy

Code Completion

Q402. Complete the code to handle a file not found error.

```
try:
f = open('missing.txt')
except ___:
print('File not found')
```

(A) IOError

(B) ValueError

[OK] (C) FileNotFoundError

(D) OSError

Correct Answer: (C)
Explanation: FileNotFoundError is raised when open() cannot find the specified file. It is a subclass of OSError.

Exception Handling

Medium

Code Completion

Q403. Complete the code to raise a custom exception with a message.

```
class InvalidAgeError(Exception):
pass

def set_age(age):
if age < 0:
___ InvalidAgeError('Age cannot be negative')
return age
```

(A) throw

(B) error

[OK] (C) raise

(D) except

Correct Answer: (C)

Explanation: 'raise' explicitly throws an exception. raise InvalidAgeError(msg) creates and raises the custom exception.

Exception Handling

Hard

Code Completion

Q404. Complete the code to chain exceptions.

```
try:
x = int('abc')
except ValueError as e:
raise RuntimeError('Conversion failed') ___ e
```

(A) because

(B) caused_by

[OK] (C) from

(D) with

Correct Answer: (C)
Explanation: 'raise X from Y' chains exceptions. Y becomes the __cause__ of X, preserving the full error context.

File I/O

Easy

Code Completion

Q405. Complete the code to read a file using a context manager.

```
___ open('data.txt', 'r') as f:
content = f.read()
```

(A) using

(B) open

[OK] (C) with

(D) file

Correct Answer: (C)
Explanation: The 'with' statement ensures the file is properly closed when the block exits, even if an exception occurs.

File I/O

Medium

Code Completion

Q406. Complete the code to write a list of lines to a file.

```
lines = ['line1\n', 'line2\n', 'line3\n']
with open('output.txt', 'w') as f:
f.___(lines)
```

(A) write

(B) append

[OK] (C) writelines

(D) output

Correct Answer: (C)
Explanation: writelines() writes a list of strings to the file. Unlike write(), it accepts an iterable of strings.

Modules

Easy

Code Completion

Q407. Complete the import to use only the sqrt function.

```
___ math ___ sqrt
print(sqrt(16)) # 4.0
```

(A) import, use

(B) load, get

[OK] (C) from, import

(D) get, from

Correct Answer: (C)
Explanation: 'from math import sqrt' imports only the sqrt function directly into the namespace.

Modules

Medium

Code Completion

Q408. Complete the code to dynamically import a module by name.

```
import ___
mod = ___.import_module('json')
data = mod.loads('{"key": 1}')
print(data)
```

(A) sys

(B) os

[OK] (C) importlib

(D) inspect

Correct Answer: (C)
Explanation: importlib.import_module(name) dynamically imports a module by string name at runtime.

Regular Expressions

Easy

Code Completion

Q409. Complete the code to find all digits in a string.

```
import re
text = 'abc123def456'
numbers = re.___(r'\d+', text)
print(numbers) # ['123', '456']
```

(A) match

(B) search

[OK] (C) findall

(D) split

Correct Answer: (C)
Explanation: re.findall() returns a list of all non-overlapping matches of the pattern in the string.

Regular Expressions

Hard

Code Completion

Q410. Complete the regex to match a valid IPv4 address.

```
import re
pattern = r'^(\d{1,3}\.){3}___$'
print(bool(re.match(pattern, '192.168.1.1')))
```

(A) \d+

(B) \d{1}

[OK] (C) \d{1,3}

(D) \d*

Correct Answer: (C)
Explanation: Each octet is 1-3 digits. The pattern matches three octets followed by dots, then a final 1-3 digit group.

Built-ins

Easy

Code Completion

Q411. Complete the code to get the sum of a list.

```
numbers = [1, 2, 3, 4, 5]
total = ___(numbers)
print(total) # 15
```

(A) add

(B) total

[OK] (C) sum

(D) count

Correct Answer: (C)
Explanation: sum(iterable) returns the sum of all elements. sum([1,2,3,4,5]) = 15.

Q412. Complete the code to apply a function to each element and get a list.

Built-ins

Medium

Code Completion

```
numbers = [1, 2, 3, 4]
squares = list(___(lambda x: x**2, numbers))
print(squares) # [1, 4, 9, 16]
```

(A) apply

(B) each

[OK] (C) map

(D) transform

Correct Answer: (C)
Explanation: map(func, iterable) applies the function to each element. Wrapping in list() materializes the results.

Q413. Complete the code to reduce a list to a single value.

Built-ins

Hard

Code Completion

```
from functools import ___
result = ___(lambda a, b: a * b, [1, 2, 3, 4])
print(result) # 24
```

(A) fold

(B) accumulate

[OK] (C) reduce

(D) combine

Correct Answer: (C)
*Explanation: functools.reduce applies the function cumulatively: ((1*2)*3)*4 = 24.*

Q414. Complete the code to create a named tuple.

Collections

Easy

Code Completion

```
from collections import ___
Point = ___('Point', ['x', 'y'])
p = Point(3, 4)
print(p.x)
```

(A) NamedDict

(B) TypedTuple

[OK] (C) namedtuple

(D) DataClass

Correct Answer: (C)
Explanation: namedtuple('Name', fields) creates a new tuple subclass with named fields.

Q415. Complete the code to create a default dictionary with list values.

Collections

Medium

Code Completion

```
from collections import ___
dd = ___(list)
dd['fruits'].append('apple')
print(dd)
```

(A) OrderedDict

(B) ChainMap

[OK] (C) defaultdict

(D) Counter

Correct Answer: (C)
Explanation: defaultdict(list) creates empty lists for missing keys, so dd['fruits'] creates a new list automatically.

Q416. Complete the code to get the 3 most common elements.

Collections

Hard

Code Completion

```
from collections import Counter
data = ['a', 'b', 'a', 'c', 'a', 'b']
c = Counter(data)
print(c.___(3))
```

(A) top

(B) frequent

[OK] (C) most_common

(D) best

Correct Answer: (C)
Explanation: Counter.most_common(n) returns the n most common elements and their counts as a list of tuples.

Q417. Complete the code to modify the outer variable from the inner function.

Scope

Medium

Code Completion

```
def outer():
count = 0
def inner():
___ count
count += 1
inner()
return count
```

(A) global

(B) outer

[OK] (C) nonlocal

(D) shared

Correct Answer: (C)
Explanation: 'nonlocal' declares that count refers to the variable in the enclosing scope, allowing modification.

Q418. Complete the closure to create a multiplier function.

Closures

Medium

Code Completion

```
def make_multiplier(n):
def multiplier(x):
return x ___ n
return multiplier
double = make_multiplier(2)
print(double(5)) # 10
```

(A) +

(B) -

[OK] (C) *

(D) /

Correct Answer: (C)
*Explanation: x * n multiplies the argument by the enclosed value n. double(5) = 5 * 2 = 10.*

Q419. Complete the code to fix the late binding issue in a loop closure.

Closures

Hard

Code Completion

```
funcs = []
for i in range(3):
funcs.append(lambda ___: x)
print([f() for f in funcs]) # [0, 1, 2]
```

(A) i=i

[OK] (B) x=i

(C) x, i=i

(D) x=i, y=x

Correct Answer: (B)
Explanation: Using a default argument x=i captures the current value of i at each iteration, fixing the late binding issue.

Q420. Complete the code to create a data class.

Advanced Python

Medium

Code Completion

```
from dataclasses import ___

@___
class Employee:
name: str
salary: float
```

(A) data

(B) model

[OK] (C) dataclass

(D) struct

Correct Answer: (C)

Explanation: @dataclass auto-generates __init__, __repr__, __eq__ and other methods based on the annotated fields.

Q421. Complete the code to implement __enter__ and __exit__ for a context manager.

Advanced Python

Hard

Code Completion

```
class Timer:
def ___(self):
import time
self.start = time.time()
return self
def __exit__(self, *args):
import time
self.elapsed = time.time() - self.start
```

(A) __init__

(B) __start__

[OK] (C) __enter__

(D) __begin__

Correct Answer: (C)
Explanation: __enter__ is called when entering the 'with' block. It sets up resources and returns the context value.

Q422. Complete the code to use the walrus operator.

Advanced Python

Hard

Code Completion

```
data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
filtered = [x for x in data if (n ___ x * 2) > 10]
print(filtered, n)
```

(A) ==

(B) =

[OK] (C) :=

(D) ->

Correct Answer: (C)
*Explanation: The walrus operator := assigns x*2 to n inside the expression. It allows assignment within an if condition.*

Decorators

Easy

Code Completion

Q423. Complete the code to apply the decorator to the function.

```
def uppercase(func):
def wrapper():
return func().upper()
return wrapper

___uppercase
def greet():
return 'hello'
print(greet()) # HELLO
```

(A) #

(B) !

[OK] (C) @

(D) $

Correct Answer: (C)
Explanation: The @ symbol is the decorator syntax. @uppercase is equivalent to greet = uppercase(greet).

Q424. Complete the code to create an iterator from a list.

Iterators

Easy

Code Completion

```
lst = [10, 20, 30]
it = ___(lst)
print(next(it)) # 10
```

(A) list

(B) next

[OK] (C) iter

(D) range

Correct Answer: (C)
Explanation: iter() creates an iterator from an iterable. next() then retrieves values one at a time.

Q425. Complete the code to create an infinite cycle over elements.

Iterators

Hard

Code Completion

```
from itertools import ___
c = ___([1, 2, 3])
result = [next(c) for _ in range(7)]
print(result) # [1, 2, 3, 1, 2, 3, 1]
```

(A) repeat

(B) chain

[OK] (C) cycle

(D) count

Correct Answer: (C)
Explanation: itertools.cycle(iterable) creates an infinite iterator that repeats the elements of the iterable.

Data Types

Hard

Code Completion

Q426. Complete the code to use precise decimal arithmetic.

```
from ___ import Decimal
result = Decimal('0.1') + Decimal('0.2')
print(result) # 0.3
```

(A) math

(B) numbers

[OK] (C) decimal

(D) fractions

Correct Answer: (C)
Explanation: The decimal module provides Decimal for exact decimal arithmetic, avoiding float precision issues.

Scope

Easy

Code Completion

Q427. Complete the code to access a global variable inside a function.

```
x = 10
def modify():
___ x
x = 20
modify()
print(x) # 20
```

(A) local

(B) outer

[OK] (C) global

(D) nonlocal

Correct Answer: (C)
Explanation: 'global x' declares that x refers to the module-level variable, allowing the function to modify it.

Q428. Complete the code to create a dictionary with default values using comprehension.

Dicts

Hard

Code Completion

```
keys = ['a', 'b', 'c']
d = {k: ___ for k in keys}
print(d) # {'a': 0, 'b': 0, 'c': 0}
```

(A) None

(B) ''

[OK] (C) 0

(D) []

Correct Answer: (C)
Explanation: The comprehension creates a dict with each key mapped to 0. dict.fromkeys(keys, 0) achieves the same.

Q429. Complete the code to find elements in either set but not both.

Sets

Hard

Code Completion

```
a = {1, 2, 3}
b = {2, 3, 4}
result = a.___(b)
print(result) # {1, 4}
```

(A) difference

(B) intersection

[OK] (C) symmetric_difference

(D) union

Correct Answer: (C)
Explanation: symmetric_difference returns elements in either set but not in both. Equivalent to a ^ b.

Tuples

Medium

Code Completion

Q430. Complete the code to count occurrences of a value in a tuple.

```
t = (1, 2, 2, 3, 2, 4)
result = t.___(2)
print(result) # 3
```

(A) find

(B) index

[OK] (C) count

(D) search

Correct Answer: (C)
Explanation: tuple.count(value) returns the number of times the value appears. 2 appears 3 times.

File I/O

Hard

Code Completion

Q431. Complete the code to read a file using pathlib.

```
from pathlib import Path
content = Path('data.txt').___
print(content)
```

(A) open()

(B) load()

[OK] (C) read_text()

(D) get_content()

Correct Answer: (C)
Explanation: Path.read_text() reads the entire file content as a string. It automatically handles opening and closing.

Q432. Complete the code to reload a previously imported module.

Modules

Hard

Code Completion

```
import importlib
import mymodule
# ... modify mymodule.py on disk ...
importlib.___(mymodule)
```

(A) reimport

(B) refresh

[OK] (C) reload

(D) update

Correct Answer: (C)
Explanation: importlib.reload(module) re-reads and re-executes the module's source file, updating its namespace.

Q433. Complete the code to create a dict mapping numbers to their cubes.

Comprehensions

Medium

Code Completion

```
cubes = {x: ___ for x in range(1, 6)}
print(cubes) # {1: 1, 2: 8, 3: 27, 4: 64, 5: 125}
```

(A) x * 2

(B) x ** 2

[OK] (C) x ** 3

(D) x * 3

Correct Answer: (C)
*Explanation: x ** 3 computes the cube of x. The dict comprehension maps each number to its cube.*

Built-ins

Medium

Code Completion

Q434. Complete the code to check if all elements are positive.

```
numbers = [1, 2, 3, 4, 5]
result = ___(x > 0 for x in numbers)
print(result) # True
```

(A) any

(B) every

[OK] (C) all

(D) each

Correct Answer: (C)
Explanation: all() returns True if every element of the iterable is truthy. All numbers are > 0 so result is True.

Regular Expressions

Medium

Code Completion

Q435. Complete the code to replace all whitespace with underscores.

```
import re
text = 'hello world python'
result = re.___(r'\s+', '_', text)
print(result) # hello_world_python
```

(A) match

(B) search

[OK] (C) sub

(D) split

Correct Answer: (C)
Explanation: re.sub(pattern, replacement, string) replaces all matches. \s+ matches whitespace sequences.

Generators

Medium

Code Completion

Q436. Complete the code to create a generator expression for squares.

```
gen = (x**2 ___ x ___ range(5))
print(list(gen)) # [0, 1, 4, 9, 16]
```

(A) of, from

(B) from, to

[OK] (C) for, in

(D) with, from

Correct Answer: (C)
Explanation: Generator expressions use the same syntax as list comprehensions: (expr for var in iterable).

OOP

Hard

Code Completion

Q437. Complete the code to define a read-only property.

```
class Circle:
def __init__(self, radius):
self._radius = radius
___
def radius(self):
return self._radius
```

(A) @staticmethod

(B) @classmethod

[OK] (C) @property

(D) @readonly

Correct Answer: (C)
Explanation: @property makes a method accessible as an attribute. Without a setter, it becomes read-only.

Inheritance

Hard

Code Completion

Q438. Complete the code to check the method resolution order.

```
class A: pass
class B(A): pass
class C(A): pass
class D(B, C): pass
print(D.___)
```

(A) __bases__

(B) __parents__

[OK] (C) __mro__

(D) __hierarchy__

Correct Answer: (C)
Explanation: __mro__ (Method Resolution Order) shows the order in which classes are searched for method lookup.

Exception Handling

Hard

Code Completion

Q439. Complete the code to suppress a specific exception.

```
from contextlib import ___
with ___(FileNotFoundError):
open('missing.txt')
print('Continued')
```

(A) ignore

(B) catch

[OK] (C) suppress

(D) handle

Correct Answer: (C)
Explanation: contextlib.suppress(ExcType) silently catches and ignores the specified exception type.

Q440. Complete the type hint for a function that takes a list of integers.

Advanced Python

Medium

Code Completion

```
from typing import ___
def sum_numbers(nums: ___[int]) -> int:
return sum(nums)
```

(A) Array

(B) Sequence

[OK] (C) List

(D) Collection

Correct Answer: (C)
Explanation: typing.List[int] annotates a list of integers. In Python 3.9+, you can use list[int] directly.

Q441. Complete the code to create a closure that adds a prefix.

Closures

Easy

Code Completion

```
def make_prefixer(prefix):
def prefixer(text):
return prefix + ___
return prefixer
add_mr = make_prefixer('Mr. ')
print(add_mr('Smith')) # Mr. Smith
```

(A) prefix

(B) make_prefixer

[OK] (C) text

(D) self

Correct Answer: (C)
Explanation: The inner function concatenates the enclosed prefix with the text argument.

Data Types

Easy

Code Completion

Q442. Complete the code to get the absolute value.

```
x = -42
result = ___(x)
print(result) # 42
```

(A) positive

(B) unsigned

[OK] (C) abs

(D) math.abs

Correct Answer: (C)
Explanation: abs() is a built-in function that returns the absolute value of a number.

Functions

Hard

Code Completion

Q443. Complete the code to define a keyword-only argument.

```
def plot(x, y, ___, color='blue'):
print(f'Plotting at ({x},{y}) in {color}')
plot(1, 2, color='red')
```

(A) /

(B) **

[OK] (C) *

(D) ...

Correct Answer: (C)
*Explanation: The bare * in the parameter list forces all following parameters to be keyword-only.*

Lists

Easy

Code Completion

Q444. Complete the code to remove the element at index 1.

```
lst = ['a', 'b', 'c', 'd']
___ lst[1]
print(lst) # ['a', 'c', 'd']
```

(A) remove

(B) drop

[OK] (C) del

(D) pop

Correct Answer: (C)
Explanation: 'del lst[1]' deletes the element at index 1. pop(1) would also work but returns the removed element.

Strings

Hard

Code Completion

Q445. Complete the code to use a raw string for a Windows path.

```
path = ___'C:\Users\admin\docs'
print(path)
```

(A) b

(B) f

[OK] (C) r

(D) u

Correct Answer: (C)
Explanation: Raw strings (r'...') treat backslashes as literal characters, preventing escape sequence interpretation.

Q446. Complete the code to remove a key-value pair and return the value.

Dicts

Medium

Code Completion

```
d = {'a': 1, 'b': 2, 'c': 3}
val = d.___('b')
print(val, d) # 2 {'a': 1, 'c': 3}
```

(A) remove

(B) delete

[OK] (C) pop

(D) discard

Correct Answer: (C)
Explanation: dict.pop(key) removes the key and returns its value. It raises KeyError if the key is missing (without default).

Q447. Complete the code to convert a list of pairs to a dict using a tuple trick.

Tuples

Hard

Code Completion

```
pairs = [('a', 1), ('b', 2), ('c', 3)]
d = ___(pairs)
print(d) # {'a': 1, 'b': 2, 'c': 3}
```

(A) list

(B) tuple

[OK] (C) dict

(D) set

Correct Answer: (C)
Explanation: dict() can take an iterable of (key, value) pairs and construct a dictionary from them.

Data Types

Easy

Code Completion

Q448. Complete the code to convert an integer to a string.

```
x = 42
s = ___(x)
print(type(s)) # <class 'str'>
```

(A) int

(B) float

[OK] (C) str

(D) repr

Correct Answer: (C)
Explanation: str() converts any object to its string representation. str(42) returns '42'.

Strings

Easy

Code Completion

Q449. Complete the code to check if a string ends with '.py'.

```
filename = 'script.py'
result = filename.___(___)
print(result) # True
```

(A) startswith('.py')

(B) contains('.py')

[OK] (C) endswith('.py')

(D) find('.py')

Correct Answer: (C)
Explanation: str.endswith(suffix) returns True if the string ends with the specified suffix.

Lists

Medium

Code Completion

Q450. Complete the code to remove all occurrences of a value from a list.

```
lst = [1, 2, 3, 2, 4, 2]
result = [x for x in lst if x ___ 2]
print(result) # [1, 3, 4]
```

(A) ==

(B) >

[OK] (C) !=

(D) is

Correct Answer: (C)
Explanation: The condition x != 2 filters out all elements equal to 2, keeping only the others.

Dicts

Easy

Code Completion

Q451. Complete the code to check if a key exists in a dictionary.

```
d = {'name': 'Alice', 'age': 30}
result = 'name' ___ d
print(result) # True
```

(A) of

(B) ==

[OK] (C) in

(D) has

Correct Answer: (C)
Explanation: 'in' operator checks for key membership in a dictionary. 'name' in d returns True.

Q452. Complete the code to create a set from a list.

Sets

Medium

Code Completion

```
lst = [1, 2, 2, 3, 3, 3]
unique = ___(lst)
print(len(unique)) # 3
```

(A) list

(B) tuple

[OK] (C) set

(D) frozenset

Correct Answer: (C)
Explanation: set(lst) creates a set from the list, removing duplicates. The result has 3 unique elements.

Q453. Complete the code to make a function accept keyword-only arguments.

Functions

Medium

Code Completion

```
def connect(host, port, ___, timeout=30):
print(f'{host}:{port} timeout={timeout}')
connect('localhost', 8080, timeout=10)
```

(A) /

(B) **

[OK] (C) *

(D) ...

Correct Answer: (C)
*Explanation: A bare * forces all following parameters to be keyword-only. timeout must be passed as a keyword.*

OOP

Medium

Code Completion

Q454. Complete the code to check if an object is an instance of a class.

```
class Animal:
pass
class Dog(Animal):
pass
d = Dog()
print(___(d, Animal)) # True
```

(A) type

(B) istype

[OK] (C) isinstance

(D) isclass

Correct Answer: (C)
Explanation: isinstance(obj, cls) returns True if obj is an instance of cls or a subclass of cls.

Inheritance

Medium

Code Completion

Q455. Complete the code to prevent a class from being instantiated directly.

```
from abc import ABC, abstractmethod
class Shape(___):
@abstractmethod
def area(self):
pass
```

(A) object

(B) Interface

[OK] (C) ABC

(D) Base

Correct Answer: (C)
Explanation: Inheriting from ABC (Abstract Base Class) and having abstract methods prevents direct instantiation.

Q456. Complete the code to create a decorator that measures execution time.

Decorators

Hard

Code Completion

```
import time
from functools import wraps
def timer(func):
@wraps(func)
def wrapper(*args, **kwargs):
start = time.___()
result = func(*args, **kwargs)
end = time.___()
print(f'{func.__name__} took {end-start:.4f}s')
return result
return wrapper
```

(A) clock

(B) now

[OK] (C) perf_counter

(D) date

Correct Answer: (C)
Explanation: time.perf_counter() provides the highest resolution timer for measuring short durations.

Q457. Complete the code to create a generator that yields only unique elements.

Generators

Medium

Code Completion

```
def unique(iterable):
seen = ___()
for item in iterable:
if item not in seen:
seen.add(item)
yield item
```

(A) list

(B) dict

[OK] (C) set

(D) tuple

Correct Answer: (C)
Explanation: A set provides O(1) membership testing with seen.add() for tracking already-yielded elements.

Q458. Complete the code to get the next value from an iterator with a default.

Iterators

Easy

Code Completion

```
it = iter([])
result = ___(it, 'default')
print(result) # default
```

(A) iter

(B) get

[OK] (C) next

(D) value

Correct Answer: (C)
Explanation: next(iterator, default) returns default instead of raising StopIteration when the iterator is exhausted.

Exception Handling

Medium

Code Completion

Q459. Complete the code to catch multiple exception types.

```
try:
x = int('abc')
except (ValueError, ___) as e:
print(f'Error: {e}')
```

(A) IOError

(B) KeyError

[OK] (C) TypeError

(D) RuntimeError

Correct Answer: (C)
Explanation: Multiple exception types can be caught in a tuple. TypeError handles type conversion issues alongside ValueError.

File I/O

Medium

Code Completion

Q460. Complete the code to create a temporary file.

```
import ___
with ___.NamedTemporaryFile(mode='w', delete=False) as f:
f.write('test data')
print(f.name)
```

(A) os

(B) io

[OK] (C) tempfile

(D) shutil

Correct Answer: (C)
Explanation: tempfile.NamedTemporaryFile creates a temporary file with a visible name in the filesystem.

Modules

Medium

Code Completion

Q461. Complete the code to list all attributes of a module.

```
import math
attributes = ___(math)
print('sqrt' in attributes) # True
```

(A) list

(B) attrs

[OK] (C) dir

(D) vars

Correct Answer: (C)
Explanation: dir(module) returns a list of names defined in the module, including functions and constants.

Regular Expressions

Medium

Code Completion

Q462. Complete the code to make the pattern case-insensitive.

```
import re
result = re.findall(r'hello', 'Hello HELLO hello', re.___)
print(len(result)) # 3
```

(A) MULTILINE

(B) DOTALL

[OK] (C) IGNORECASE

(D) VERBOSE

Correct Answer: (C)
Explanation: re.IGNORECASE makes the pattern match regardless of case. All three variations of 'hello' are matched.

Collections

Hard

Code Completion

Q463. Complete the code to create a ChainMap for layered lookups.

```
from collections import ___
defaults = {'color': 'red', 'size': 10}
overrides = {'color': 'blue'}
config = ___(overrides, defaults)
print(config['color'], config['size'])
```

(A) OrderedDict

(B) defaultdict

[OK] (C) ChainMap

(D) Counter

Correct Answer: (C)
Explanation: ChainMap searches dictionaries in order. overrides is checked first, so color='blue', then defaults gives size=10.

Scope

Hard

Code Completion

Q464. Complete the code to access the built-in list function after it has been shadowed.

```
list = [1, 2, 3] # shadows built-in
import ___
real_list = ___.list
print(real_list(range(3))) # [0, 1, 2]
```

(A) sys

(B) os

[OK] (C) builtins

(D) types

Correct Answer: (C)
Explanation: The builtins module contains all built-in names. Importing it gives access to the original list type.

Closures

Hard

Code Completion

Q465. Complete the code to inspect closure variables.

```
def make_counter(start):
count = start
def counter():
nonlocal count
count += 1
return count
return counter
c = make_counter(10)
print(c.___[0].cell_contents)
```

(A) __dict__

(B) __vars__

[OK] (C) __closure__

(D) __scope__

Correct Answer: (C)

Explanation: __closure__ is a tuple of cell objects containing the closed-over variables. cell_contents gives the actual value.

Advanced Python

Hard

Code Completion

Q466. Complete the code to create a class using type() directly.

```
MyClass = ___('MyClass', (object,), {'x': 10, 'greet': lambda self: 'hi'})
obj = MyClass()
print(obj.x, obj.greet())
```

(A) class

(B) object

[OK] (C) type

(D) meta

Correct Answer: (C)
Explanation: type(name, bases, dict) dynamically creates a class. This is what class statements do internally.

Topic-wise Breakdown

Topic	Count	Topic	Count	Topic	Count
Functions	29	Data Types	28	Strings	28
Lists	26	Dicts	26	Built-ins	26
OOP	25	Advanced Python	23	Exception Handling	21
Generators	20	Sets	19	File I/O	19
Regular Expressions	19	Inheritance	18	Decorators	18
Modules	18	Collections	18	Tuples	17
Iterators	17	Comprehensions	17	Scope	17
Closures	17				

Question Type Summary

MCQ	Output	Scenario	Code Completion
186 questions	96 questions	92 questions	92 questions

Generated by Expert QA Interviewer | Python MCQ Set | All Rights Reserved